# Full-Stack Web Development of an Automated Assessment Platform for UML Diagrams

**Project ID: CCDS25-0233**

Submitted in Partial Fulfilment of the Requirements
for the Degree of Bachelor of Computer Science
of the Nanyang Technological University

by

Wang Yu Teng (U2122796L)

Supervisor: Dr. Loke Yuan Ren

Examiner: Prof. Zhang Hanwang

College of Computing and Data Science

2026

# Abstract

The rapid growth of computer science enrollments has intensified the demand for scalable assessment tools, yet grading Unified Modelling Language (UML) diagrams remains a labour-intensive manual process. UML diagrams require evaluation on both structural correctness and semantic alignment with design principles, and their open-ended nature admits multiple valid solutions, a characteristic that has historically resisted automation. Traditional automated approaches are tightly coupled to specific tooling, restricted to class diagrams, and unable to accommodate variation in student responses. Recent studies have demonstrated the promise of Large Language Models (LLMs) for UML grading but were conducted with earlier-generation models carrying notable limitations, and no prior work has embedded LLM-based grading within a fully engineered, deployable assessment platform.

This project addresses both gaps. It presents a full-stack web-based assessment platform built in TypeScript that supports the end-to-end assessment workflow: course management, question pool construction, assignment configuration, student submission collection, and orchestrated grading with grade publication. The platform adopts a three-tier architecture. It consists of a React frontend, a Hono API server, and a Graphile Worker background process. These components are containerised using Docker and deployed via Kubernetes. A custom UML editor captures student diagrams in a structured format exported to both PlantUML text and rendered images, enabling a dual-modality grading strategy. The pipeline enforces structured output through Zod schema validation and implements a human-in-the-loop review workflow where AI suggestions require explicit instructor approval.

A formal experiment evaluated six LLMs: one flagship and one baseline from OpenAI, Anthropic, and Google, on 32 UML class diagram submissions across five rubric criteria over 960 API calls. Claude Opus 4.6 achieved the highest grading accuracy (Pearson r = 0.92, MAE = 0.68/10) and, together with Claude Sonnet 4.6, the highest weighted aggregate score (0.87) across accuracy, consistency, cost, speed, and structured output compliance. To the best of the author's knowledge, this work represents one of the first implementations that integrates structured-output LLM grading within a full-stack assessment platform with human-in-the-loop review.

# Acknowledgements

I would like to begin by expressing my heartfelt appreciation to my supervisor, Dr. Loke Yuan Ren, for his unwavering guidance, encouragement, and mentorship throughout the course of this project. I am especially grateful for his remarkable patience and understanding in throughout the course of this project. His expertise and invaluable advice were instrumental in bringing this project to fruition, and the experience has sharpened my capabilities in engineering, research, and problem-solving in ways I will carry forward well beyond this degree.

I would also like to thank the faculty and staff of Nanyang Technological University and the College of Computing and Data Science for shaping my undergraduate journey. Their dedication to teaching and genuine passion for the discipline helped cultivate and sustain my enthusiasm for Computer Science and Software Engineering.

# Table of Contents

**Abstract**.....2
**Acknowledgements**.....4
**Table of Contents**.....5
**List of Tables**.....7
**List of Figures**.....8
**1. Introduction**.....9
1.1 Background and Motivation.....9
1.2 Problem Statement.....10
1.3 Objectives.....12
1.4 Scope of the Project.....13
1.5 Significance of the Study.....14
1.6 Report Organization.....15
**2. Literature Review**.....16
2.1 Traditional Approaches to Automated UML Assessment.....16
2.2 LLM-Based Approaches to Automated Assessment.....18
2.2.1 LLMs for UML Diagram Assessment.....18
2.2.2 LLMs for Broader Educational Assessment.....20
2.3 Assessment Platforms.....21
2.3.1 Commercial Assessment Platforms.....22
2.3.2 Academic Assessment Platforms.....22
2.4 UML Modelling Tools and Diagram Representation.....24
2.5 Summary and Research Gaps.....25
**3. Design and Integration**.....26
3.1 System Architecture.....26
3.1.1 High-Level Architecture.....26
3.1.2 Technology Stack.....28
3.1.3 Client-Side Architecture.....30
3.2 Database Design.....31
3.2.1 Schema Overview.....31
3.2.2 Key Schema Decisions.....33
3.3 API Design.....34
3.3.1 Route Structure and Response Format.....34
3.3.2 Middleware Chain.....35
3.3.3 Authentication and Authorisation.....35
3.4 Assessment Lifecycle Implementation.....37

3.4.1 Course Creation and Enrollment....37
3.4.2 Question Pool Construction....38
3.4.3 Assignment Configuration....39
3.4.4 Student Assessment Attempt....40
3.4.5 Submission Processing....42
3.4.6 Grading and Grade Publication....43
3.5 LLM-Based Automated Grading Pipeline....45
3.5.1 Asynchronous Job Architecture....45
3.5.2 Prompt Design....46
3.5.3 UML Diagram Grading Strategy....47
3.5.4 Structured Output and Defensive Validation....48
3.5.5 Human-in-the-Loop Review....49
3.6 Manual Grading Interface....49
3.6.1 Two-Panel Layout....49
3.6.2 Per-Question Grading Form....50
3.6.3 Submission Navigation....51
3.7 UML Diagram Editor....52
3.8 Deployment and Infrastructure....54
3.8.1 Container Strategy....54
**4. Experiment and Results....57**
4.1 Research Questions....57
4.2 Experiment Design....57
4.2.1 Model Selection....57
4.2.2 Dataset....58
4.2.3 Grading Rubric....60
4.2.4 Prompt and Pipeline Design....60
4.2.5 Evaluation Metrics....60
4.3 Results....61
4.3.1 Grading Accuracy (RQ1)....61
4.3.2 Cost-Performance Tradeoff (RQ2)....62
4.3.3 Consistency Across Runs (RQ3)....63
4.3.4 Structured Output Compliance (RQ4)....63
4.3.5 Response Speed....64
4.3.6 Normalization and Aggregate Ranking....65
4.4 Discussion....67
**5. Conclusion....69**
**6. Future Work....71**
**Bibliography....73**

# List of Tables

Table 3.1 Technology Stack ..... 29

Table 3.2 Question type content structures and associated grading methods ..... 38

Table 4.1 Models evaluated in the experiment ..... 58

Table 4.2 Synthetic submission quality tiers ..... 59

Table 4.3 Evaluation metrics and weights ..... 60

Table 4.4 Grading accuracy - Pearson correlation and MAE against ground truth ..... 61

Table 4.5 Cost per submission and accuracy for each model ..... 62

Table 4.6 Inter-run consistency ..... 62

Table 4.7 Structured output compliance and rubric adherence ..... 63

Table 4.8 Median response latency per submission (seconds) ..... 64

Table 4.9 Weighted aggregate ranking ..... 66

# List of Figures

Figure 3.1 High-level architecture diagram showing three-tier structure ..... 27

Figure 3.2 Entity-Relationship Diagram ..... 33

Figure 3.3 Staff Dashboard showing the course grid with course cards ..... 37

Figure 3.4 Question Pool management page ..... 39

Figure 3.5 Assignment Builder Modal ..... 39

Figure 3.6 Student Assessment Attempt UI ..... 41

Figure 3.7 Submission processing state machine diagram showing transitions ..... 43

Figure 3.8 Screenshot of the Staff Auto-Grading Dashboard ..... 44

Figure 3.9 Sequence diagram showing the batch AI grading flow ..... 46

Figure 3.10 Two-panel grading interface ..... 49

Figure 3.11 UML diagram editor ..... 52

Figure 3.12 Docker multi-stage build pipeline ..... 54

Figure 3.13 Kubernetes deployment topology ..... 54

Figure 3.14 CI/CD pipeline stages ..... 55

# 1. Introduction

## 1.1 Background and Motivation

Over the past decade, computer science (CS) enrollments have more than doubled in many universities [1]. The increasing popularity of CS education has resulted in operational obstacles when it comes to managing assessments [2]. Together with the rapid expansion of online learning, the demand for scalable and effective tools to assess CS students' work has intensified [3]. Automated assessment platforms (AASPs) have long been explored in programming contexts, where correctness of a student's program can be validated using test cases. Among these various automated assessments, evaluating Unified Modelling Language (UML) diagrams presents unique difficulties.

UML is a well-known modelling language used in software design [4]. Software engineers typically utilise UML diagrams to communicate and understand code architecture, system design and proposed implementations [4], [5]. Although UML is no longer as widely adopted and closely followed in the industry [6], it is still common in universities' CS curricula to teach students [7]. UML diagrams, such as use case, class, sequence, activity and state machine diagrams are used extensively to teach object-oriented design, software architecture and systems analysis [8]. Yet, despite their pedagogical importance, UML diagram assignments are particularly time-consuming and difficult to grade. UML diagrams are judged not only for syntactic and structural correctness but also for semantic alignment with design principles and intended system behaviour.

This poses a major challenge to AASPs because evaluating UML diagrams requires subjective judgement and allows multiple valid answers. Therefore, universities rely on manually marking UML diagrams which are typically done by teaching assistants (TAs) and professors. This approach introduces challenges in both grading consistency and operational cost [9]. A single cohort of two hundred students submitting an assignment that contains five UML diagram questions produces one thousand individual diagrams requiring grading. Even at a conservative pace of 5 minutes per diagram, that translates to approximately eighty-three hours of grading effort which is a huge burden to university staff. The subjectivity inherent in marking UML further exacerbates the problem as different markers may interpret the same grading rubric criterion differently. This inconsistency poses a recognised threat to assessment validity [10]. Thus, these challenges have motivated a body of research into automating UML assessment, yet existing approaches remain limited in scope and generalizability.

## 1.2 Problem Statement

Prior research has investigated traditional methods for automating UML assessment such as rule-based checking, machine learning techniques and algorithmic matching [11], [12], [13]. While these solutions successfully establish the feasibility of automating UML grading, they are tightly coupled with specific rule sets, limited to a few types of UML diagrams, and fail to handle more sophisticated diagrams. These automated grading approaches do not generalize across diagram types [2], [14] and cannot accommodate the natural variation in valid student responses to an open-ended design problem [14].

More recently, research has shown significant promise in using Large Language Models (LLMs) for assessing UML by converting the diagrams to natural language text and applying grading

rubrics [2]. Likewise, other work has demonstrated that LLMs can assess UML diagrams in an educational context through well-defined prompts [15]. However, these studies were conducted with an earlier generation of LLMs that carried notable limitations: smaller context windows constrained the complexity of diagrams, weaker reasoning capabilities limited the depth of semantic analysis, and stochastic outputs led to grading inconsistencies [15]. Crucially, the LLM and Application Programming Interface (API) ecosystem has undergone rapid evolution. State-of-the-art (SOTA) models offer substantially larger context windows, stronger reasoning [16], and native support for structured output enforcement through API-level features [17]. These capabilities directly address the consistency and versatility concerns raised in earlier works, yet no existing study has revisited LLM-based UML assessments.

At the institutional level, universities require more than an automated grading pipeline. They require a complete, deployable platform that manages the full lifecycle of an assessment [3]. This includes the organization of courses, construction of question pools, configuration of assignments with appropriate access and time controls, the collection and storage of student submissions, the orchestration of automated grading processes and the review and adjudication of scores. Building such a platform demands not only LLM expertise but also substantial software engineering effort in full-stack web development, cloud computing, database design and DevOps. Therefore, this project aims to bridge both gaps by re-evaluating LLM-based UML automated assessment by leveraging current SOTA models and embedding the grading pipeline within a fully engineered assessment platform. The remainder of this chapter formalises these objectives and defines the project scope.

## 1.3 Objectives

The primary objective of this project is to develop a complete assessment platform that leverages current SOTA models and their surrounding tooling to deliver automated grading of UML diagrams that is both reliable across diverse UML diagram types and consistent and reproducible. To achieve the primary objective, the following secondary objectives are required:

[**O1**] To design and develop a full-stack web-based assessment platform that supports the end-to-end assessment lifecycle, encompassing course management, question pool construction, assignment configuration with access and time controls, student submission collection and automated grading orchestration.

[**O2**] To implement an LLM-based automated grading pipeline that evaluates student-submitted UML diagrams against instructor-defined rubrics, leveraging structured output enforcement to produce consistent and deterministic grading results.

[**O3**] To design the platform with the explicit goal of reducing the time burden on TAs and professors across the entire assessment workflow. Where automated grading is applied, instructors should be able to audit, adjust and finalise scores through a streamlined review user interface (UI). Where manual grading is required, the platform should provide a structured and efficient grading experience that minimises effort.

[**O4**] To conduct a systematic evaluation of multiple LLM models to identify the most suitable candidate for automated UML grading, assessing each model on rubric adherence, structured output compliance, cost-efficiency, consistency across repeated runs and alignment with human TA grading.

[**O5]** To ensure that the platform is readily deployable on NTU's infrastructure through containerisation and to ensure that the platform can handle the load of concurrent student submissions during live assessments in controlled environments.

## 1.4 Scope of the Project

The scope of this project encompasses the design, implementation and empirical evaluation of a full-stack web-based assessment platform tailored to NTU's CS education. The platform addresses the complete assessment lifecycle and comprises the following key components:

- **Assessment Management -** Staff may create courses, question pools and assignments. Assignments are configurable with specific access windows and time limits that enforce a controlled assessment environment.
- **Multi-format Question Support -** The platform supports 3 distinct question types: MCQ, written questions and UML diagram questions.
- **LLM-Assisted Automated Grading -** For written and UML diagram question types, the platform integrates an LLM-based automated pipeline that evaluates student submissions against instructor-defined rubrics.
- **Deployment and Infrastructure -** The system is designed for on-premise deployment, packaged as a multi-container Docker application orchestrated using Kubernetes.
- **Empirical Evaluation of LLM models -** A formal experiment is conducted comparing multiple LLM models, including both open-source and commercial models. The LLMs are evaluated based on grading accuracy, rubric adherence, structured output compliance, consistency and cost-effectiveness.

The platform does not support:

- **Coding questions -** as it requires additional significant development work to implement and is well-addressed by existing tools such as CodeRunner [18].
- **IP/network restrictions -** as it can be handled by the institution's networking system or controlled browser environment.
- **Plagiarism detection -** as the main use-case of the platform is to administer assignments under a controlled environment and supervision.

## 1.5 Significance of the Study

The most significant contribution of this project is the reduction in time and effort required from TAs during the manual grading of UML diagrams. By automating the initial grading pass through LLMs and presenting results through a streamlined review UI, the platform enables instructors to use their time on more valuable tasks such as providing qualitative feedback. Beyond time savings, the platform also addresses grading consistency. A LLM-based grading pipeline applies the same rubric interpretation uniformly across all student submissions.

From a technical standpoint, this project contributes a timely empirical evaluation of current SOTA LLM models for UML diagram assessment. Prior studies in this area were conducted with earlier-generation models that carried notable limitations as stated in section 1.2. By revisiting UML grading with current models and leveraging modern API features, this study provides an updated empirical dataset and findings. Furthermore, to the best of my knowledge, this is one of the first fully deployed full-stack assessment platform combining structured-output LLM grading using SOTA models with a complete assessment lifecycle.

## 1.6 Report Organization

The remainder of this report is organised in the following way: Chapter 2 reviews related work on automated UML assessment, LLM-based grading, assessment platforms and UML representation formats, identifying the gaps that motivate this project. Chapter 3 describes the system design and implementation, covering architecture, database schema, API design, the LLM grading pipeline and deployment infrastructure. Chapter 4 presents the formal experiment comparing six LLM models for UML grading accuracy. Chapter 5 synthesises the project's contributions and limitations, and Chapter 6 outlines directions for future work.

# 2. Literature Review

This chapter surveys the existing body of literature relevant to the automated assessment of UML diagrams and the engineering of assessment platforms. This review is organized thematically into four areas. Section 2.1 examines traditional, non-LLM approaches to automating UML grading. Section 2.2 explores emerging use of LLMs for education assessment, with particular attention to UML-specific studies. Section 2.3 reviews the landscape of automated assessment platforms and the software engineering considerations that underpin them. Section 2.4 surveys UML modelling tools and diagram representation formats that inform how student submissions are captured and converted for automated processing. Finally, Section 2.5 synthesizes the identified gaps and positions this project within the current body of work.

## 2.1 Traditional Approaches to Automated UML Assessment

The manual grading of UML diagrams has long been recognised as a bottleneck in CS education. A single class diagram can take an instructor up to fifteen minutes to evaluate [19] and the process is compounded further by the inherent subjectivity of UML problems, where multiple structurally distinct solutions may be equally valid [13]. This has motivated a substantial body of research into automated or semi-automated grading techniques.

One of the earliest systems to address this problem was UMLGrader [12]. UMLGrader accepts student-submitted class diagrams via a web interface and compares them to a reference solution using heuristics such as flexible identifier matching and multiplicity checking, providing immediate feedback that supports iterative resubmission [12]. However, it is explicitly designed for tightly constrained problems that test notation mastery, and it supports only class diagrams

[12]. Its reliance on IBM Rational Rose as the input tool further restricted its practical use in modern curricula.

Bian et al. [13] advanced the field in 2019 by proposing an algorithm that integrates syntactic, semantic and structural matching strategies. Syntactic matching employs Levenshtein distance to handle spelling errors, while semantic matching leverages WordNet to recognise synonyms. Structural matching detects more nuanced student choices such as class splits, merges and misplaced attributes. The approach was validated on a real classroom assignment involving twenty students, achieving grading results within fourteen percent of the instructor baseline. Despite these advances, the approach remains limited to class diagrams and has not been extended to other UML diagram types [13].

Subsequent work by Modi et al. [19] in 2021 developed a Java-based tool that evaluates UML diagrams by comparing submissions in XMI format. The system parses XMI files exported from the Modelio modelling environment, performs the comparison, allocates marks, and generates separate feedback files for both students and tutors. While the tool supports multiple UML diagram types in principle, its practical validation was limited to only class diagrams, and its strict dependency on Modelio restricts the tooling available to students [19].

More recently, Jebli et al. [20] in 2023 proposed an automated solution that similarly exports student and tutor diagrams via Modelio into XMI, parses them with the Document Object Model (DOM) API, and matches elements across thirteen categories such as classes, attributes, and operations. The system was able to deduct marks from student diagrams, correctly attributed to missing elements. However, the system assumes the tutor's diagram to be the gold standard, which means alternative valid designs that differ structurally would still be penalised [20].

Taken together, these traditional approaches share a common set of characteristics. They provide immediate, deterministic and scalable feedback that is free from human grader fatigue and subjectivity. However, they are tightly coupled to specific rule sets and UML tools, limited to class diagrams, and fundamentally incapable of handling the semantic richness and natural variation inherent in open-ended design problems [2], [13], [20]. These limitations provided the impetus for researchers to explore more flexible approaches, particularly those leveraging large language models, as discussed in the following section.

## 2.2 LLM-Based Approaches to Automated Assessment

### 2.2.1 LLMs for UML Diagram Assessment

The application of LLMs to UML assessment is a recent and rapidly evolving research direction. Wang et al. [15] conducted one of the first studies in this area, investigating whether ChatGPT (specifically GPT-4o) could assess use case, class, and sequence diagrams in an educational context. Forty student UML reports were graded by both the LLM and human experts against eleven evaluation criteria. The results showed that ChatGPT could assess UML diagrams but generally scored lower than human graders by an average difference of approximately 1.18 points out of 11. Performance varied by diagram type: the model was strongest on use case diagrams, but exhibited weaknesses on class and sequence diagrams due to frequent misunderstanding of evaluation criteria and misidentification of elements [15]. Key limitations included the use of only a single LLM, a small dataset of forty students, and the stochastic nature of LLM output where the same input may produce different grades across runs [15].

Building on this foundation, Bouali et al. [2] in 2025 conducted the first empirical comparison of multiple LLMs for UML class diagram grading. The study evaluated GPT o1-mini, Claude Sonnet, and Llama 3.2 alongside a sentence-transformer semantic similarity baseline, using ninety-two student submissions of constrained class diagrams. The methodology involved converting student diagrams from JSON into natural language descriptions and applying a forty-point rubric to both human teaching assistants and LLMs. Claude Sonnet emerged as the best performer with an intraclass correlation coefficient (ICC) of 0.76 against human graders and the lowest error metrics among all models. The study demonstrated that with careful configuration and structured rubrics, LLMs can approach parity with human grading. However, the authors noted that LLMs may miss deeper conceptual nuances or creative edge cases, and advocated for a human-in-the-loop approach where automated tools complement rather than replace human graders [2].

The DUET (Diagrammatic UML & ER Tutor) system, introduced by Gürtl et al. [21] in 2025, uses LLM-based assessment as a formative feedback tool rather than a summative grading mechanism. DUET employs a two-stage pipeline where GPT-4o first converts uploaded student and reference diagrams into PlantUML, and a smaller LLM (Mistral-7B) then compares the two representations to generate feedback. An evaluation with two educators and four TAs highlighted benefits in accessibility and scalability, but also raised concerns about the non-deterministic nature of outputs, which undermines trust for formal assessment purposes [21]. Prior work cited by the DUET authors reported that GPT-4 achieved approximately ninety-one percent accuracy on attributes but only twenty-five percent on class relationships, further underscoring the challenge of achieving reliable, production-grade grading [21]. The study acknowledges that

LLM-based assessment has not yet reached the reliability threshold required for high-stakes automated assessment.

### 2.2.2 LLMs for Broader Educational Assessment

Beyond UML-specific studies, a growing body of literature examines how LLMs perform in broader educational grading contexts, and these findings are directly relevant to the design of any LLM-based grading pipeline.

Zhang et al. [22] in 2024 evaluated ChatGPT 3.5, ChatGPT 4.0, and Claude 3 Haiku on criterion-based grading using IELTS academic writing tasks. The study found that LLMs achieved good interrater consistency with human graders (ICC = 0.89) but only moderate absolute agreement, meaning they were reliable in ranking students relative to one another but systematically biased in absolute scores, typically grading lower than official human examiners. A key finding was that domain-specific knowledge embedded in prompts, specifically the provision of detailed evaluation rubrics, mattered more than model size or complexity. Interestingly, teachers without formal training showed worse consistency than the LLMs, highlighting the potential of LLMs as stable, scalable feedback providers when human expertise is scarce [22]. The study recommends against using LLM scores as absolute grades, instead advocating their use for tracking learning growth against defined criteria.

In the domain of programming assessment, Piscitelli et al. [23] in 2025 evaluated LLMs (GPT-3.5, GPT-4o, Claude Sonnet, and Llama 3) on 350 C programming assignments using their ClueLLM4Eval tool. Claude Sonnet in one-shot mode achieved the closest alignment with human grading at an average difference of 1.17 points out of 10, with approximately sixty percent of scores falling within ±1 point. However, format compliance was a significant

challenge: many models produced outputs that did not conform to the required grading format, necessitating post-processing validators and parsers [23]. The study emphasised that LLMs alone are insufficient for reliable self-assessment and that human-in-the-loop or hybrid approaches are necessary.

Taking a different approach, Xie et al. [24] in 2024 proposed a multi-agent system called Grade-Like-a-Human that models the entire human grading workflow. Rather than treating rubrics as static inputs, the system uses LLMs to iteratively refine rubrics based on both the question and sampled student answers. Multiple prompting strategies are applied during grading, followed by a post-grading review step where group comparison and re-grouping detect anomalies and ensure consistency. The system outperformed baseline automated short-answer grading methods, especially for complex, multi-step questions [24]. For this project, the concept of rubrics improving LLMs' performance is particularly relevant.

Across these broader studies, several consistent themes emerge. First, structured rubrics and carefully engineered prompts are decisive factors in LLM grading performance [22], [23], [24]. Second, LLMs are more reliable for relative ranking and consistency than for absolute scoring [22]. Third, output format compliance and determinism remain practical challenges that require engineering solutions such as structured output enforcement and validation layers [23].

## 2.3 Assessment Platforms

Automating the grading of student work is only one component of the broader challenge. Universities require complete, deployable platforms that manage the entire lifecycle of an assessment, from course organisation through to score adjudication [29]. This section reviews the

landscape of assessment platforms, both commercial and academic, and the engineering considerations that underpin them.

### 2.3.1 Commercial Assessment Platforms

Several commercial platforms dominate the automated assessment space. HackerEarth provides coding challenges, and skill assessments with gamification features such as leaderboards, but its assessment engine is limited to code execution and multiple-choice questions with no support for modelling exercises like UML diagrams [25]. Similarly, Codility and LeetCode focus on algorithmic coding assessments through test-case-based dynamic analysis [26], [27]. Blackboard, a widely adopted Learning Management System (LMS), supports general assignment submission and rubric-based manual grading but provides no automated grading capabilities for diagrammatic content [28]. A common limitation across all these commercial platforms is their exclusive focus on textual or code-based submissions; none offers native support for UML diagram capture, automated comparison, or LLM-based assessment.

### 2.3.2 Academic Assessment Platforms

In the academic sphere, Mekterovic et al. [29] introduced Edgar, a comprehensive Automated Programming Assessment System (APAS) that supports multiple programming paradigms including C, Java, SQL, and MongoDB. Edgar's primary contribution is the identification and integration of nine feature categories that define a comprehensive APAS: course administration, monitoring, logging, problem mitigation, analysis, data import/export, content authoring, question types and grading, and user-friendly online testing [29]. These feature categories serve as guiding principles for our project's implementation.

The scaling challenges of such platforms were further examined by Mekterovic et al. [30] in a companion study that focused on the architectural bottlenecks encountered during six years of operating Edgar. Dynamic analysis (executing untrusted student code), static analysis, and plagiarism detection were identified as the three primary scalability challenges. The study advocates for polyglot persistence (combining PostgreSQL for structured data with MongoDB for transient data), horizontal scaling of code runner clusters, and the implementation of resource-intensive modules such as static analysis and plagiarism detection as standalone, generic services [30]. These architectural patterns are instructive for any assessment platform that must handle concurrent submissions and computationally expensive grading operations.

Artemis, developed at the Technical University of Munich, is perhaps the most comprehensive open-source assessment platform currently available [31]. It supports programming exercises, modelling exercises, text exercises, and file upload exercises. Artemis includes a semi-automated assessment workflow for UML modelling exercises that combines automatic suggestions (based on machine learning cluster analysis of previous submissions) with manual grading. Instructors can define grading criteria, provide inline feedback on specific diagram elements, and review submissions through a structured assessment UI[31]. While Artemis represents the closest existing system to a UML assessment platform, its auto-assessment for modelling exercises relies on clustering similarity rather than LLM-based semantic evaluation, and the platform's overall complexity makes it challenging to deploy and customise for specific institutional needs. The capabilities of any assessment platform are, however, constrained by the format in which student work is captured, which motivates a closer examination of UML modelling tools.

## 2.4 UML Modelling Tools and Diagram Representation

A critical design decision for any UML assessment platform is the choice of diagram authoring tool, as it determines the format of student submissions and consequently the feasibility of automated processing. Traditional desktop UML tools such as Modelio [32] provide comprehensive modelling capabilities and export to the standardised XMI format, which has been the basis for several of the traditional grading approaches reviewed in Section 2.1 [12], [19]. However, their heavyweight desktop-only architectures, steep learning curves, and installation requirements make them poorly suited for lightweight, web-based assessment environments where students need to complete exercises within a browser [32].

Web-based general-purpose diagramming tools such as Draw.io (Diagrams.net) [33] and MermaidJS [34] offer greater accessibility. Draw.io provides a drag-and-drop UI with UML shape libraries and exports to XML, SVG, and PNG formats. MermaidJS uses a text-based syntax to define diagrams and renders them in the browser, making it well-suited for version control and automated processing. However, neither tool enforces UML semantics or provides structured output that encodes the model's elements in a machine-readable format suitable for direct comparison.

Apollon [35], developed at the Technical University of Munich as part of the Artemis ecosystem, is a UML modelling editor written in React and TypeScript. It provides an infinite canvas, supports a wide range of UML diagram types (class, use case, activity, sequence, communication, object, deployment, and component diagrams), and is available as an NPM package that can be embedded directly into JavaScript applications. Apollon produces a structured JSON model of the diagram that encodes all elements, relationships, and their

properties, making it highly suitable for automated processing [35]. Its modern web technology stack, embeddable architecture, and broad diagram type coverage make it a strong candidate for integration into a web-based assessment platform.

The choice of diagram representation format has direct implications for the grading pipeline. Image-based submissions (PNG, SVG) require a conversion step either via optical recognition or LLM vision capabilities which introduces potential parsing errors, as noted in several studies [2], [16]. Structured formats such as XMI, Umple DSL, or Apollon's JSON eliminate this conversion step but constrain the choice of authoring tool. Text-based intermediate representations such as PlantUML have emerged as a pragmatic bridge: DUET [21] uses GPT-4o to convert diagram images into PlantUML before analysis, while the approach in [2] converts JSON representations into natural language descriptions.

## 2.5 Summary and Research Gaps

Traditional approaches to UML grading have demonstrated the feasibility of automation but remain fundamentally limited by their tight coupling to specific tools, their restriction to class diagrams, and their inability to handle the semantic variation inherent in open-ended design problems [2], [13], [20]. LLM-based approaches have shown significant promise: commercial LLMs can grade very similarly to human graders [2] and can process multiple UML diagram types [15]. However, the existing studies were conducted with earlier-generation models that carried limitations in context window size, reasoning depth, and output consistency [15], and no study has revisited these findings with SOTA models that offer substantially larger context windows, stronger reasoning capabilities, and native support for structured output enforcement through API-level features such as tool use and JSON schemas. At the platform level, existing

automated assessment systems such as Edgar [29] and Artemis [31] provide robust engineering foundations for programming assessments, but none integrates LLM-based grading for UML diagrams within a complete, deployable assessment lifecycle. Artemis comes closest but its auto-assessment relies on clustering rather than LLM-based evaluation. Commercial platforms such as HackerEarth, Codility, and LeetCode do not support diagrammatic assessment at all. Thus, the present project aims to bridge these gaps by re-evaluating LLM-based UML assessment using current SOTA models and embedding the grading pipeline within a fully engineered, full-stack assessment platform.

# 3. Design and Integration

This chapter presents the design, architecture and implementation of the UML Assessment Platform. The platform is a full-stack web application that manages the end-to-end assessment lifecycle, from course creation and question pool construction through to student submission, automated grading and grade publication. Each section discusses the design decisions, trade-offs and implementation details that support the project objectives defined in Section 1.3.

## 3.1 System Architecture

### 3.1.1 High-Level Architecture

The platform follows a monolithic Single-page Application (SPA) plus API-server architecture with a separated background worker process. As seen in figure 3.1, three logical tiers interact to serve the assessment workflow: a React single-page application running in the browser, a Hono HTTP server running on Node.js, and a standalone Graphile Worker process that executes

long-running LLM grading jobs asynchronously. Both the web server and the worker connect to a managed PostgreSQL database hosted on Supabase, which additionally provides authentication services. The LLM grading jobs call external APIs from OpenAI and Anthropic.

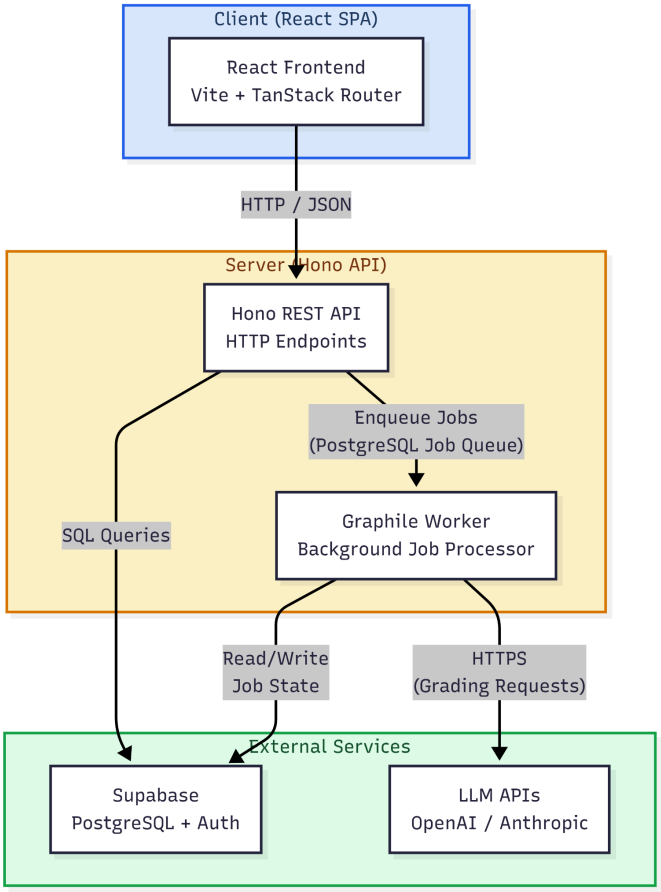


*Figure 3.1: High-level architecture diagram showing three-tier structure*

A monolithic backend was chosen over microservices because a single codebase with modular route files yields one Docker image and one deployment unit. This is a practical fit for school-scale deployments maintained by IT staff without distributed-systems experience. The codebase is organised into eleven route modules, each corresponding to a domain resource (courses, assignments, submissions, etc.), achieving the maintainability benefits of modular code

without the operational overhead of inter-service communication, service discovery and distributed tracing.

The critical architectural decision is the separation of the web server and background worker into independent processes. LLM grading calls can take up to a few minutes per question, and executing them within the HTTP request cycle would block the event loop and degrade response times for all concurrent users. By decoupling job enqueueing from job processing through PostgreSQL, the web server remains responsive during exam-period traffic spikes while the worker processes the grading queue independently. This separation also enables independent scaling: the web tier scales horizontally using a Kubernetes Horizontal Pod Autoscaler (HPA) to handle concurrent student submissions during examinations, while the worker tier scales manually based on grading queue depth. Crucially, the web server's only responsibility at submission time is to insert a row into the jobs table and return an acknowledgement, an operation that completes in single-digit milliseconds regardless of queue depth, so the student experience during the critical submission window is entirely decoupled from the computational cost of LLM inference, achieving [O5] (handle the load of concurrent student submissions during live assessments).

### 3.1.2 Technology Stack

The platform is built entirely in TypeScript, providing end-to-end type safety. Table 3.1 summarises the core technology choices.

| Layer | Technology |
|---|---|
| **Language** | TypeScript 5.9 |
| **Runtime** | Node.js 20 |

| Layer | Technology |
|---|---|
| **HTTP Framework** | Hono 4.11 |
| **Database** | PostgreSQL (via Supabase) |
| **Frontend** | React 19 |
| **Authentication** | Supabase Auth + custom Json Web Token (JWT) |
| **Background Jobs** | Graphile Worker 0.16 |
| **AI Integration** | Vercel AI SDK 6.0 |
| **Validation** | Zod 4.3 |
| **UML Editor** | xyflow 12.10 + plantuml-encoder |
| **Testing** | Vitest + Playwright |
| **Container** | Docker (multi-stage, Node 20 Alpine) |
| **Orchestration** | Kubernetes / k3s |

*Table 3.1 - Technology Stack*

Several technology choices warrant justification as they directly impact the project objectives:

**Graphile Worker -** Graphile Worker uses PostgreSQL itself as the job queue, leveraging advisory locks for concurrency control. This eliminates the need for a Redis instance, reducing infrastructure complexity. For a school deployment that already requires PostgreSQL, this means zero additional services to deploy and maintain. Furthermore, because jobs are stored in PostgreSQL, they are transactional with the application data. A grading job's result and the associated mark record can be committed atomically, preventing split-brain inconsistencies between the job queue and the application database.

**Vercel AI SDK -** The SDK provides a unified interface across LLM providers. This abstraction enables the platform to switch between OpenAI and Anthropic models via configuration alone, without code changes. The SDK's API with Zod schemas enforces structured, validated grading

output, directly supporting the structured output enforcement requirement of **[O2]** (implementing accurate, structured LLM-based automated grading).

**Supabase -** The decision to use Supabase rather than running PostgreSQL in-cluster or using an alternative managed provider was driven by the combination of services Supabase bundles under a single platform. Supabase provides managed PostgreSQL with daily automatic backups and point-in-time recovery, a complete authentication service, and Row Level Security (RLS) policies that enforce access control at the database level. Alternatives such as Neon offer managed PostgreSQL but would require a separate authentication solution, adding infrastructure to maintain. By consolidating database, authentication and connection pooling into a single managed service, Supabase reduces the number of external dependencies from three to one, simplifying both the deployment architecture, development and operational burden. The primary trade-off is vendor coupling and the requirement for outbound internet connectivity from the Kubernetes cluster to Supabase's hosted infrastructure.

### 3.1.3 Client-Side Architecture

The frontend is a client-rendered SPA built with React 19 and served as static files by the Hono server. The decision to use client-side rendering rather than a server-side rendering framework such as Next.js was driven by the observation that the entire platform sits behind authentication. There are no public pages that would benefit from search engine indexing. A client-rendered SPA avoids the complexity of server-side rendering and keeps the build pipeline simple: Vite produces a static bundle that any web server can host. The disadvantage is a slower initial page load compared to server-rendered alternatives. In practice, this is acceptable because users access the platform only after authentication, so the initial load occurs once per session.

Client-side state is managed using TanStack Query, which provides a declarative cache layer between the React components and the API server. Every API resource is associated with a query key; when a mutation succeeds (for example, a teacher publishes grades), TanStack Query automatically invalidates the relevant query keys and refetches stale data. This approach avoids the boilerplate of a global state store such as Redux, while ensuring that data displayed to users is never more than one polling interval out of date.

## 3.2 Database Design

### 3.2.1 Schema Overview

The schema is organised around five functional groups that mirror the platform's domain boundaries, a deliberate alignment that ensures each group can evolve independently as requirements change. The sixteen tables are distributed across the 5 groups as follows:

The **identity and access group** (users, passwordResetTokens, refreshTokens, enrollments) manages authentication state and course membership.

The **assessment authoring group** (courses, questions, assignments, assignmentQuestions, rubrics) defines what students are assessed on; questions belong to a course-level pool and are linked to assignments through the assignmentQuestions junction table, enabling a single question to be reused across multiple assignments.

The **student work group** (submissions, answers) captures the artefacts produced during an assessment attempt: a submission record is created when a student begins an assignment and tracks lifecycle metadata such as tab-switch events, while individual answers are stored with a

composite unique constraint on (submissionId, questionId) to guarantee exactly one response per question per attempt.

The **grading grou**p (marks, aiGradingJobs) records both automated and manual outcomes, tracking whether the grade originated from the LLM pipeline, from manual review, or from a teacher overriding an AI suggestion, preserving a full audit trail of grading provenance.

Finally, the **platform operations group** (aiUsageStats, staffNotifications, systemSettings) supports day-to-day administration: aggregated cost statistics, real-time notifications and a key-value settings store for global configuration. The schema is defined using Drizzle ORM's declarative TypeScript API, from which sequential SQL migrations are generated and applied.

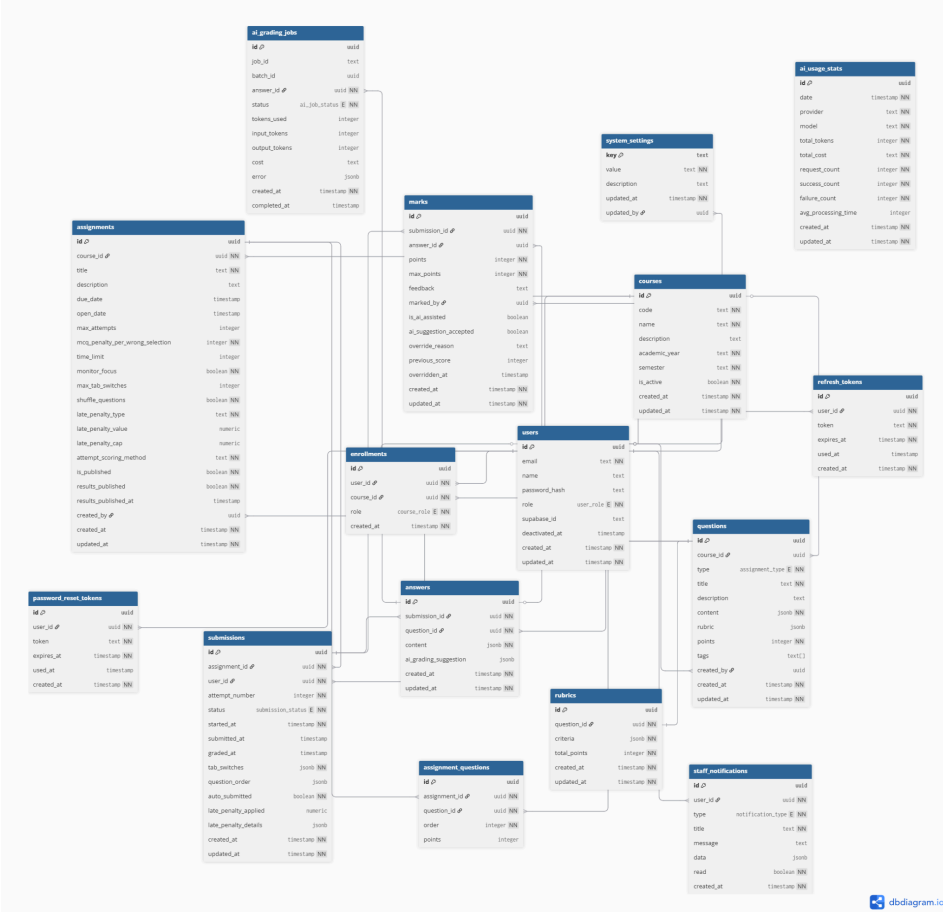


*Figure 3.2. Entity-Relationship Diagram*

### 3.2.2 Key Schema Decisions

**JSONB for polymorphic content -** The platform supports three question types: MCQ, written, and UML - each with fundamentally different data shapes. Rather than normalising these into three separate content tables with complex polymorphic joins, the `questions` and `answers` tables use JSONB columns for their `content` fields. This keeps the schema manageable at fifteen tables while PostgreSQL's JSONB operators allow targeted queries when needed. The trade-off is reduced compile-time type safety for JSONB fields. Content extraction relies on runtime utility functions validated by Zod schemas rather than TypeScript inference.

**PostgreSQL enums for state machines -** The `submission_status` enum (`draft → submitted → late → grading → graded`) and `ai_job_status` enum (`pending → processing → completed → failed`) enforce valid state transitions at the database level. This prevents application-level bugs from creating invalid states such as a submission that is simultaneously `draft` and `graded`. The main drawback is that adding new states requires a database migration, but state machine changes in an assessment platform should be deliberate and reviewed.

**Composite unique constraints for data integrity** - Three critical constraints prevent data corruption under concurrent access: `enrollments(userId, courseId)` ensures one enrolment per student per course; `answers(submissionId, questionId)` prevents duplicate answers; `assignmentQuestions(assignmentId, questionId)` prevents duplicate question bindings. These constraints are essential to the platform's data integrity guarantees under concurrent examination conditions (objectives **[O1]** and **[O5]**). Concurrent HTTP requests can produce duplicate records if integrity is enforced only at the application level, particularly under the network conditions of a large examination hall where clients may retry failed requests. By encoding these uniqueness invariants as database-level constraints, the platform guarantees that even in the presence of application-level bugs or retry storms, the assessment data remains consistent.

## 3.3 API Design

### 3.3.1 Route Structure and Response Format

The server exposes eighty-three endpoints organised into eleven route modules, each corresponding to a domain resource. All endpoints return a consistent JSON response shape, enabling a single error-handling path on the client side where the `apiClient()` wrapper checks the `success` field before processing the response. Request validation is enforced at the route boundary using Zod schemas. Every request body and query parameter set is validated before the handler logic executes. If validation fails, a 400 response is returned immediately, guaranteeing that handler code always operates on correctly typed inputs.

### 3.3.2 Middleware Chain

The Hono middleware pipeline applies security headers, CORS restrictions, body-size limits, rate limiting, and JWT authentication in a strict, fail-fast order: each layer rejects invalid requests before the next layer executes, so an unauthenticated request is never rate-limited and a rate-limited request never reaches the database. Rate limits are tuned per endpoint sensitivity: 1,000 requests per 15 minutes globally, with tighter limits on authentication endpoints to mitigate credential-stuffing. The global threshold provides approximately 67% headroom above a worst-case examination burst of 200 students submitting concurrently.

### 3.3.3 Authentication and Authorisation

The platform implements a dual JWT validation strategy. When a request arrives, the auth middleware first attempts to verify a custom JWT. If that fails, it falls back to Supabase session verification. This dual approach supports both cloud deployments where Supabase handles the session management natively and on-premise deployments where Supabase may be unreachable. On first Supabase login, a user record is automatically created in the `users` table with a role

inferred from their email domain. The role inference function maps NTU email patterns to platform roles: `@e.ntu.edu.sg` maps to student, `@staff.main.ntu.edu.sg` maps to staff, and a hardcoded list of administrator emails receives the admin role. This domain-based inference simplifies onboarding by eliminating a manual role-assignment step, but it couples the platform's access control to NTU's email naming convention. If the university changes its domain structure, the mapping function must be updated. The risk of privilege escalation through email spoofing is mitigated by the requirement that all accounts are created through Supabase's authenticated sign-up flow, which verifies email ownership via a confirmation link.

Authorisation operates at two levels. Global roles (admin, staff, student) are enforced by the requireRole() middleware, which gates entire route groups. Course-scoped roles (lecturer, TA, lab executive, student), stored in the enrollments table, provide finer-grained access within a course context. For example, only a lecturer or TA can trigger auto-grading.

As an additional layer of defence, Supabase's Row Level Security (RLS) policies enforce access control at the database level, independently of the application middleware. If an application-level bug bypasses the middleware or a misconfigured endpoint exposes a query without authorisation checks, the database itself rejects the unauthorised row access. This comes at a cost of operational complexity of keeping two authorisation layers consistent, but the benefit is that compromising grading data requires simultaneous failures in both layers, directly supporting the controlled environment required by **[O5]**.

## 3.4 Assessment Lifecycle Implementation

This section traces the complete assessment lifecycle consisting of six stages from course creation through grade publication, describing how each stage is implemented and how the design decisions support objectives **[O1]** and **[O3]**.

### 3.4.1 Course Creation and Enrollment

The assessment lifecycle begins when a staff member creates a course. Enrollment determines which users can interact with a course and in what capacity. The platform supports individual enrollment by email address and bulk enrollment via CSV upload. Bulk enrollment is transactional: if any email in the CSV does not correspond to an existing user, the entire operation rolls back, preventing partial enrollments that would require manual cleanup.

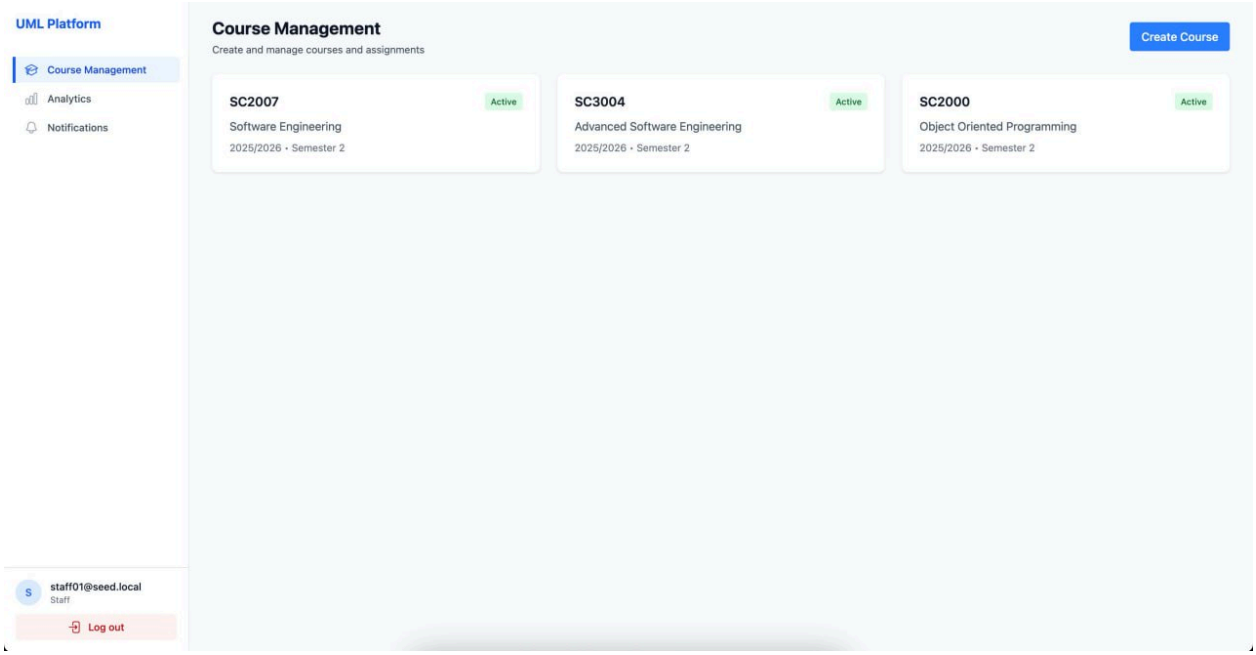


*Figure 3.3: Staff Dashboard showing the course grid with course cards.*

### 3.4.2 Question Pool Construction

Questions are scoped to a course and can be reused across multiple assignments within that course. This design separates question authoring from assignment construction: a lecturer might build up a pool of fifty questions over a semester and draw different subsets into different assessments. The platform supports three question types, each with a different content structure stored in the JSONB `content` column of the `questions` table.

| Question Type | JSONB Content Structure | Grading Method |
|---|---|---|
| MCQ | `{prompt, options: [{id, text, isCorrect}], allowMultiple}` | Automatic: exact-match against correct option flags with configurable penalty for incorrect selections |
| Written | `{prompt, modelAnswer}` | Manual or AI-assisted with rubric-based criteria scoring |
| UML | `{prompt, referenceDiagram, modelAnswer}` | AI-assisted using vision API for diagram analysis with rubric-based criteria scoring |

*Table 3.2: Question type content structures and associated grading methods*

The question pool UI provides search, type-based filtering, and tag-based filtering to help staff locate questions efficiently as the pool grows. Staff can also import or export questions in bulk via CSV files and export the pool, supporting question sharing between course coordinators

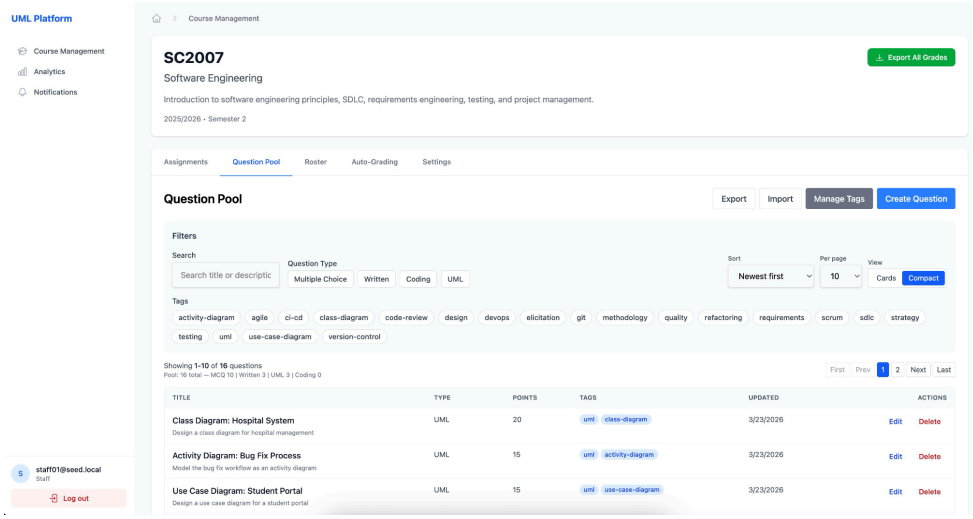


*Figure 3.4: Question Pool management page*

### 3.4.3 Assignment Configuration

Assignments are the core unit of assessment, linking questions from the course pool to an assessment event with its own timing rules, attempt policies, and grading configuration.

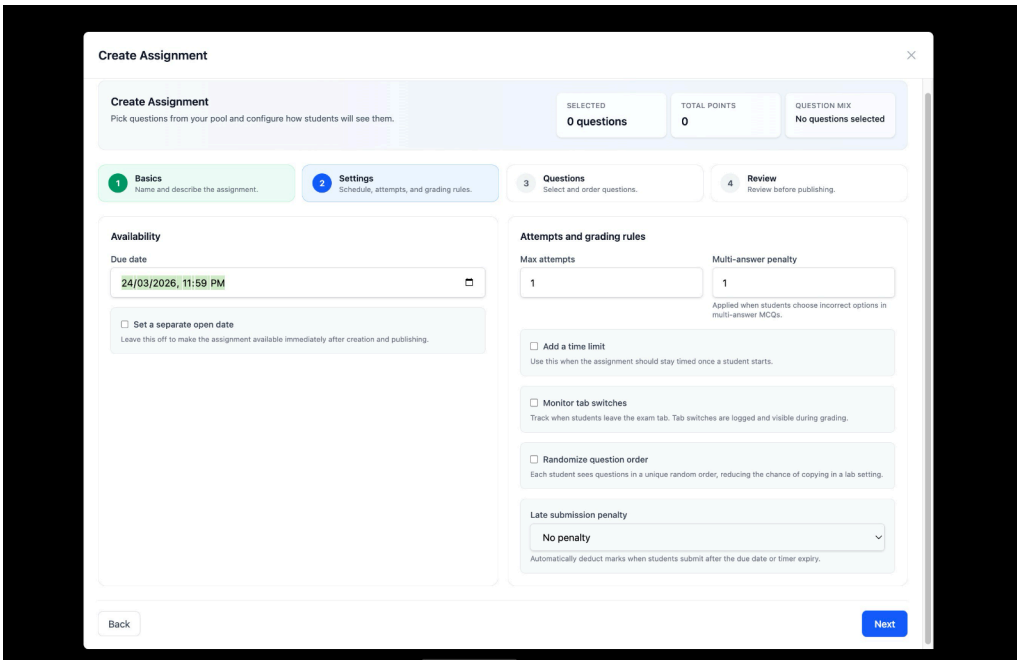


*Figure 3.5: Assignment Builder Modal*

**Timing and Attempt Controls** - Each assignment has a due date and an optional time limit. The countdown begins when the student starts their attempt, not when the assignment opens, so the effective deadline is computed as: *effectiveDeadline = min(dueDate, startedAt + timeLimit)*. Staff can also permit multiple attempts, with the final grade taken from either the latest or highest-scoring submission, determined via a configurable setting.

**Question Ordering and Focus Monitoring** - Question shuffling uses a deterministic seed derived from the student's user identifier, producing a unique but reproducible order that persists across page refreshes. This mitigates answer-sharing without sacrificing resumability, directly supporting the controlled-environment objective [O5]. Focus monitoring is an opt-in proctoring layer: staff specify a maximum number of permitted tab switches, and exceeding the threshold triggers automatic submission of the student's current work.

**Publishing** - Assignments remain in draft state, invisible to students and freely editable, until explicitly published. Publishing is a one-way transition that locks the question set and timing configuration, preventing a race condition where a student begins an attempt while the staff member simultaneously modifies the assignment

### 3.4.4 Student Assessment Attempt

The assessment attempt orchestrates answer state management, periodic auto-saving, countdown timing, focus monitoring, and submission confirmation

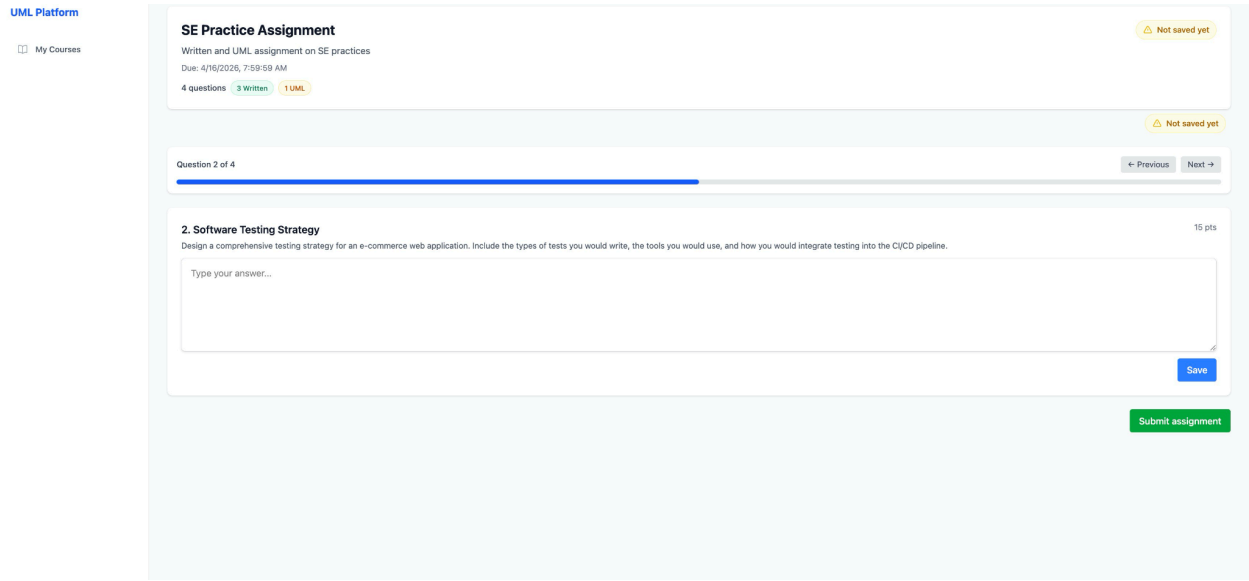


*Figure 3.6: Student Assessment Attempt UI*

**Starting and resuming an attempt** - The start endpoint is designed to be idempotent: if a draft submission already exists for a student, the server returns it with saved answers rather than creating a duplicate. This is critical for examination reliability, as browser crashes or accidental tab closures mid-attempt would otherwise block the student entirely. The question navigator supports non-linear navigation with a progress bar, allowing students to skip difficult questions and return to them later.

**Auto-save mechanism** - A periodic save triggers every thirty seconds whenever the student has unsaved changes. A thirty-second save interval balances data safety against API traffic where shorter intervals would multiply requests proportionally during peak examination windows, while longer intervals risk unacceptable data loss (for example, during a browser crash). A visible timestamp indicator gives students confidence that their work is being preserved.

**Timer Enforcement** - Timed assessments use a dual-layer enforcement strategy. The client displays a visible countdown for immediate responsiveness, but client-side timers alone are unreliable since students could manipulate the browser clock or pause execution by closing their laptop. A server-side cron job therefore polls every sixty seconds for expired drafts and force-submits them. This combination ensures no submission can remain in draft past its deadline, with a worst-case enforcement delay of sixty seconds.

**Focus Monitoring** - When enabled for an assignment, tab switches are tracked and logged. A toast warns the student on the first switch; if the count reaches the configured threshold, the system auto-submits. Focus monitoring is opt-in per assignment, recognising that different assessments serve different pedagogical purposes. A practice quiz does not require the same proctoring as a final examination.

### 3.4.5 Submission Processing

When a student submits their work whether manually, via timer expiry, or through a focus-monitoring breach, the server executes a submission pipeline wrapped in a single database transaction. This transactional approach was chosen to guarantee atomicity: a submission either completes fully or remains in draft state, eliminating the risk of partially recorded submissions that could confuse both students and staff.

A key design decision was to separate the grading of MCQ and non-MCQ question types at submission time. Because MCQ grading is deterministic, it can be resolved instantly and synchronously. This means that assignments containing only MCQ questions are graded the moment they are submitted, giving students and teachers immediate results without burdening the asynchronous grading queue. Assignments that also include written or UML questions

transition to a submitted status and await manual or AI-assisted grading. Upon completion, the server returns a submission receipt containing the timestamp, any late penalty applied, and a confirmation identifier. The client then transitions to a read-only view, providing clear visual feedback that the submission is finalised and preventing accidental edits.

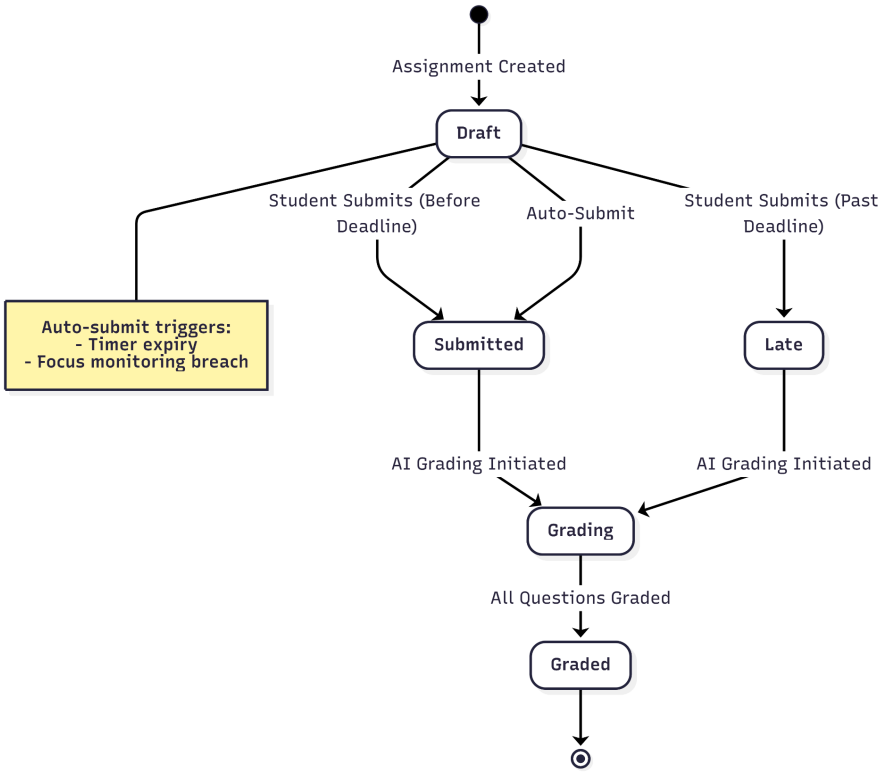


*Figure 3.7: Submission processing state machine diagram showing transitions*

### 3.4.6 Grading and Grade Publication

The platform supports three grading pathways: automatic MCQ grading at submission time, LLM-assisted grading for written and UML questions (Section 3.5), and manual grading through the review interface (Section 3.6).

A central design principle is human-in-the-loop grading. When staff trigger batch AI grading, suggestions are generated for eligible answers but are never automatically promoted to official marks. A staff member must explicitly review and accept each suggestion. Every grading action is recorded with full provenance metadata, including who assigned the mark, whether it was AI-assisted, and whether the AI suggestion was accepted or overridden. This audit trail allows the institution to verify that AI-assisted grades received appropriate human oversight, directly supporting **[O3]**.

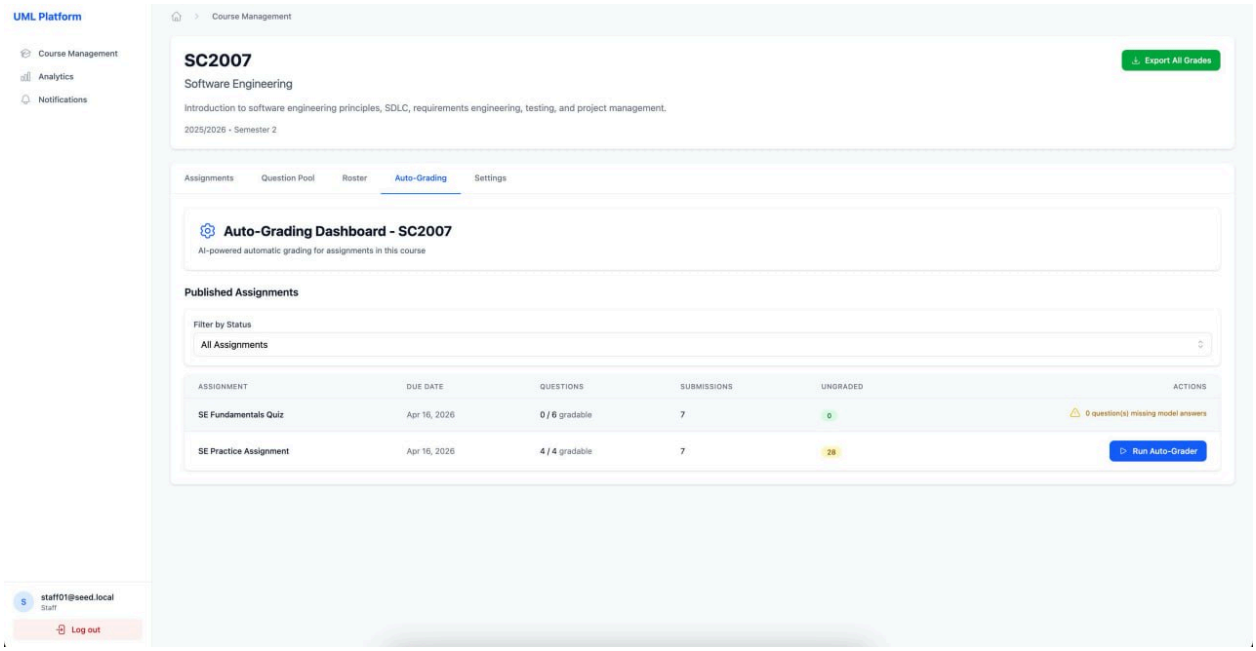


*Figure 3.8: Screenshot of the Staff Auto-Grading Dashboard*

Grade visibility is also deliberately controlled. Students see only their submission status until the staff member explicitly publishes results for an assignment. This prevents students from viewing partial or in-progress grades, avoiding confusion while the grading queue is still being worked through. Once published, students can view per-question scores, grader feedback, late penalties, and which marks were AI-assisted, maintaining transparency about how their work was evaluated.

## 3.5 LLM-Based Automated Grading Pipeline

The automated grading pipeline addresses the platform's core value proposition: reducing teacher time spent on marking UML questions while preserving grading integrity. This section describes the pipeline's design as a series of deliberate choices that balance automation against human oversight, throughput against reliability, and flexibility against complexity.

### 3.5.1 Asynchronous Job Architecture

LLM grading calls take anywhere from five to sixty seconds per question which is far too slow for synchronous HTTP responses. The pipeline therefore operates asynchronously: when staff trigger batch grading, the API server creates job records and enqueues Graphile Worker tasks, returning immediately. The worker processes jobs independently, storing results as suggestions for later review. The worker runs at a concurrency of one, processing jobs sequentially. This trades throughput for predictability as parallel LLM requests risk rate-limit errors. For the target scale of 50–100 students per course, sequential processing completes within an acceptable window. Each job follows a fetch–prompt–call–validate–store sequence. If any stage fails, the job records the error and notifies staff, ensuring failures are visible rather than silent.

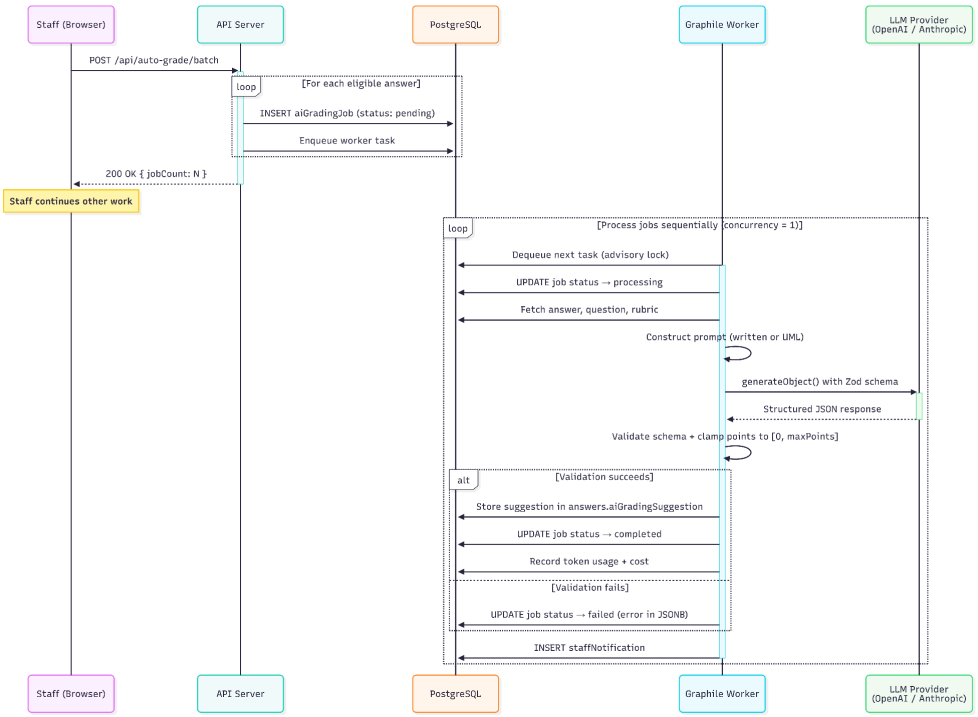


*Figure 3.9: Sequence diagram showing the batch AI grading flow*

### 3.5.2 Prompt Design

The pipeline uses separate prompt templates for written and UML questions, stored in version-controlled source code, this ensures prompt changes are reviewed and traceable through Git history, important because prompt wording directly affects grading accuracy. Both templates share a common structure: the LLM receives the question text, the model answer, the rubric criteria with point allocations, and the student's response. The prompt instructs the model to evaluate each rubric criterion independently, assign per-criterion scores with reasoning, and compute a total. This criterion-level decomposition is what enables meaningful partial credit. Without it, the model would produce a single holistic score with no transparency into how marks were distributed. For UML questions, the prompt additionally includes the student's diagram as

an image via the LLM's vision capability, alongside the lecturer's reference diagram. This multi-modal input allows the model to assess visual properties, notation correctness, relationship multiplicities, layout clarity which otherwise cannot be evaluated from source text alone.

### 3.5.3 UML Diagram Grading Strategy

Grading UML diagrams presents a distinct challenge compared to written answers: the student's response is a visual artefact with both structural semantics and spatial properties. The literature identifies several possible approaches to automated UML assessment, which were evaluated during design.

*Textual DSL conversion* (converting diagrams to structured syntax and grading the text alone) preserves structural information but discards visual properties such as notation correctness and layout clarity. *Graph matching* (encoding diagrams as adjacency structures and computing structural diffs) offers deterministic comparison but requires implementing graph-diff logic and cannot assess semantic quality such as naming conventions or design rationale. *Code generation* (converting UML to skeleton source code and comparing implementations) leverages LLM strengths in code review but introduces a lossy translation step that strips UML-specific semantics like multiplicities and stereotypes. *Pure vision* (sending only the rendered image) allows holistic assessment but risks hallucination on fine-grained details such as attribute types or relationship labels, which may be difficult to read from a rasterised image.

The platform adopts a **dual-modality approach** that combines textual and visual inputs. The UML editor stores each student's diagram in a structured format. During grading, the pipeline performs two extraction steps: it serialises the diagram's underlying data into PlantUML DSL,

producing a machine-readable text that explicitly encodes classes, attributes, methods, etc.; and it exports the rendered diagram as a PNG image. Both representations, along with the corresponding reference answer in the same dual format, are included in the LLM prompt. The PlantUML text is embedded as part of the prompt's textual content, while the images are supplied via the model's vision capability as described in Section 3.5.3.

This dual-input design addresses the weaknesses of each modality in isolation. The PlantUML text gives the LLM an unambiguous, parseable view of the diagram's structure, eliminating the risk of misreading labels or multiplicities from a rasterised image. The rendered image allows the LLM to assess visual properties that are not fully captured by the DSL. By providing both, the model can cross-reference the textual structure against the visual rendering, improving grading accuracy without requiring a separate graph-matching implementation or a UML-to-code translation step.

### 3.5.4 Structured Output and Defensive Validation

Unstructured LLM responses are unusable in an automated pipeline. The platform enforces structured output through the Vercel AI SDK's generateObject() function, which accepts a Zod schema defining the required response shape. If the model returns malformed output or omits required fields, the job fails explicitly rather than storing corrupt data. A secondary defence clamps awarded points to a specified range, guarding against the known tendency of LLMs to occasionally hallucinate out-of-range scores. Together, schema validation and score clamping ensure that every suggestion reaching the review interface is structurally sound and within valid bounds, directly supporting [O2].

### 3.5.5 Human-in-the-Loop Review

The most consequential design decision in the pipeline is that AI suggestions are never automatically promoted to official marks. Completed suggestions are stored in a staging column on the answer record, separate from the marks table. Staff must explicitly accept or reject each suggestion through the grading UI. Accepting creates a mark record with full provenance metadata: who reviewed it, that it was AI-assisted, and whether the suggestion was accepted as-is or overridden. This audit trail gives institutions verifiable evidence that every published grade received human oversight. The downside of this approach is an additional review step compared to fully autonomous grading. This was considered acceptable because reviewing a pre-populated suggestion with per-criterion reasoning is substantially faster than grading from scratch, achieving the time-saving goal [O3].

## 3.6 Manual Grading Interface

The manual grading interface is where the platform's time-saving value proposition is realised. The UI is designed to minimise the number of clicks, page transitions, and context switches required to move through a stack of student submissions.

### 3.6.1 Two-Panel Layout

The grading UI adopts a two-panel layout: a submission list on the left and a grading form on the right. This design eliminates the back-and-forth navigation that characterises traditional grading workflows, where a teacher must open a submission, grade it, return to the list, and open the next one. With both panels visible simultaneously, the staff member selects a submission from the list

and the grading form populates immediately without a page transition, keeping the full queue visible at all times so the grader maintains awareness of overall progress.

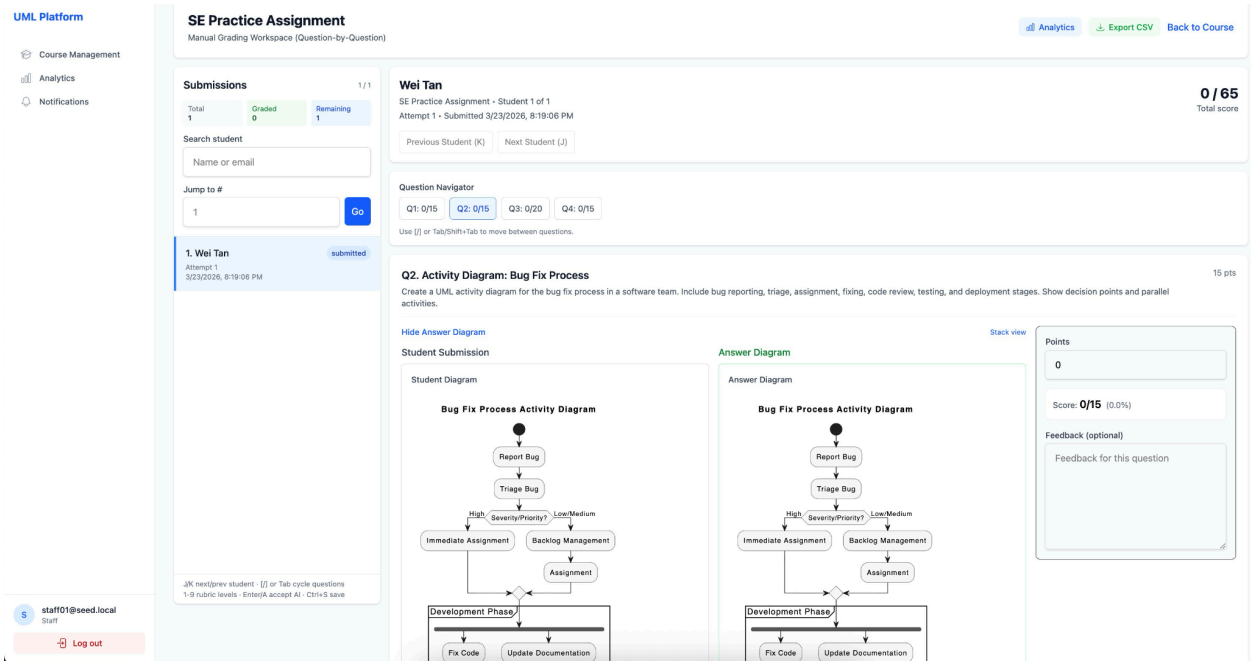


*Figure 3.10: Two-panel grading interface*

The submission list supports sorting by score, searching by student email, and filtering by grading status. These controls allow staff to triage efficiently. For example, filtering to show only ungraded submissions, or sorting by score to review outliers. A grading completion indicator on each list entry provides at-a-glance progress, showing how many questions in each submission have been marked.

### 3.6.2 Per-Question Grading Form

For each question in a submission, the grading form presents the question prompt, the student's answer, the model answer, and the rubric side by side. This co-location of reference material with

the student's work eliminates the need to switch between tabs or documents to recall what a correct answer looks like, a common source of grading slowdown in traditional workflows.

For written questions, the student's text response is displayed alongside the model answer. For UML questions, the interface renders both the student's diagram and the reference diagram visually using the same UML viewer component used during the assessment attempt, so staff see exactly what the student produced. The grading form provides a numeric score input bounded to the question's maximum points and a free-text feedback field. Rubric criteria are displayed inline so that the grader can award marks per criterion rather than assigning a single holistic score. This structure mirrors the criterion-level decomposition used by the LLM pipeline (Section 3.5.3), ensuring consistency between manual and AI-assisted grading outputs.

### 3.6.3 Submission Navigation

Previous and next buttons allow staff to move sequentially through the filtered submission list without returning to the list panel. This keyboard-navigable flow supports a heads-down grading rhythm: grade one submission, press next, grade the next, and so on. The interface preserves the active filter and sort state across navigation, so a staff member who has filtered to show only submissions with pending AI suggestions can step through that subset continuously. Combined with the pre-populated AI suggestions, this sequential flow transforms the grading task from an evaluative exercise into a verification exercise, significantly reducing the cognitive effort per submission.

## 3.7 UML Diagram Editor

The UML diagram editor is the UI through which students construct class diagrams during assessments and staff author reference diagrams in the question pool. It is built as two shared React components, `UMLEditor` for interactive editing and `UMLViewer` for read-only display. Both are reused across question authoring (Section 3.4.2), the student assessment attempt (Section 3.4.4), and the staff grading UI (Section 3.6.2). This component reuse guarantees that every stakeholder sees an identical visual representation of a diagram, eliminating ambiguity when comparing student work against reference answers.

### 3.7.1 Technology Choice

The editor is built on React Flow (xyflow v12.10), a node-and-edge canvas library that provides pan, zoom, drag-and-drop, and connection-drawing out of the box. The alternative was a custom canvas implementation, which would have required reimplementing these standard interaction primitives which require substantial effort orthogonal to the project's grading objectives. React Flow's declarative API also integrates naturally with React state and the auto-save mechanism described in Section 3.4.4. The trade-off is that React Flow is a general-purpose graph library with no built-in UML semantics. The platform therefore implements custom node types representing UML classes (with name, attribute, and method compartments) and custom edge types for each UML relationship (association, aggregation, composition, inheritance, realisation), each rendered with its correct notation.

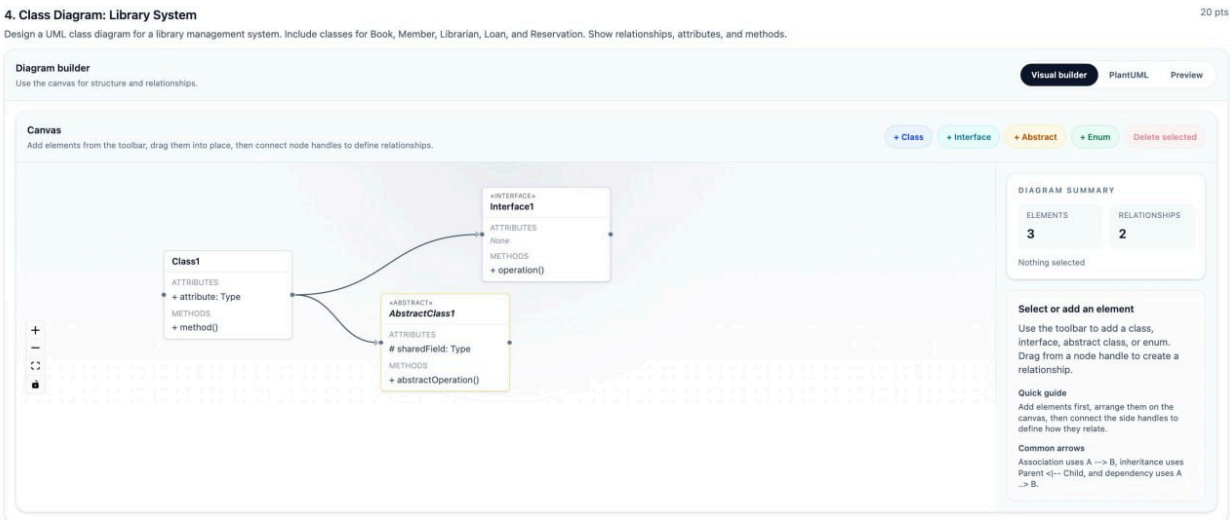


*Figure 3.11: UML diagram editor*

### 3.7.2 Structured Input over Free Text

Class members are entered through structured inline forms, selecting a visibility modifier, typing a name, and specifying a type, rather than free-text UML notation. This design decision directly supports the grading pipeline (Section 3.5.3): structured input produces machine-readable data that can be serialised to PlantUML DSL deterministically, without requiring a text parser that could introduce errors into grading input. The same principle applies to relationships, where the type and multiplicities are selected from a popover rather than typed as syntax.

### 3.7.3 Export for Grading

When the grading pipeline processes a UML answer, it invokes two export functions on the stored diagram state. The first serialises the editor state into PlantUML DSL text, mapping each class node to a PlantUML class block with its attributes and methods, and each edge to the corresponding PlantUML relationship syntax. The second generates a PNG image URL via the PlantUML server encoding. Both outputs are embedded in the LLM prompt as described in

Section 3.5.3. This export step is deterministic, the same editor state always produces the same PlantUML text and image, ensuring that grading results are reproducible.

## 3.8 Deployment and Infrastructure

The deployment strategy supports both cloud-hosted demonstration and on-premise deployment.

### 3.8.1 Container Strategy

The platform is packaged as a single Docker image using a multi-stage build on Node 20 Alpine. The first stage compiles TypeScript and bundles the frontend; the second copies only production dependencies and compiled output, producing a minimal image. The same image serves both the web server and the background worker (Section 3.5.1): the default entrypoint starts the Hono HTTP server, while an overridden command starts the Graphile Worker instead. This halves the number of builds to maintain while preserving the independent scaling described in Section 3.1.1. The container runs as a non-root user for defence-in-depth.

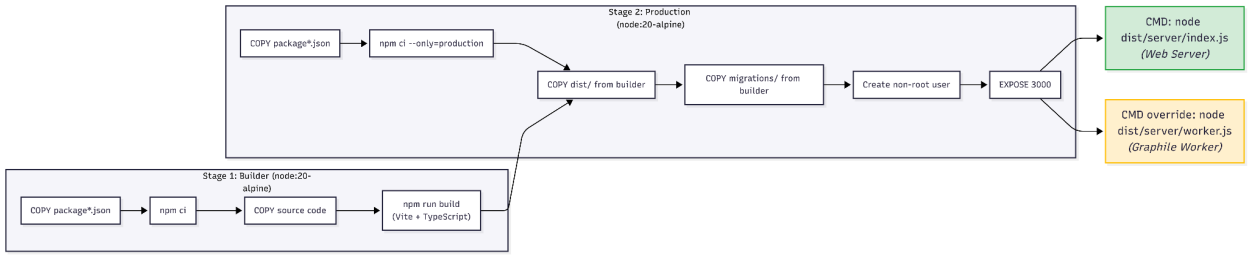


*Figure 3.12: Docker multi-stage build pipeline.*

### 3.8.2 Kubernetes Orchestration

For production school deployments, Kubernetes manifests define separate deployments for the web server and worker, each using the same image with different command overrides. The web

tier scales horizontally via HPA to handle examination traffic spikes and the worker runs as a single replica to avoid LLM rate-limit contention. Database migrations run as a one-off job before application pods start, ensuring the schema is current before any request is served.

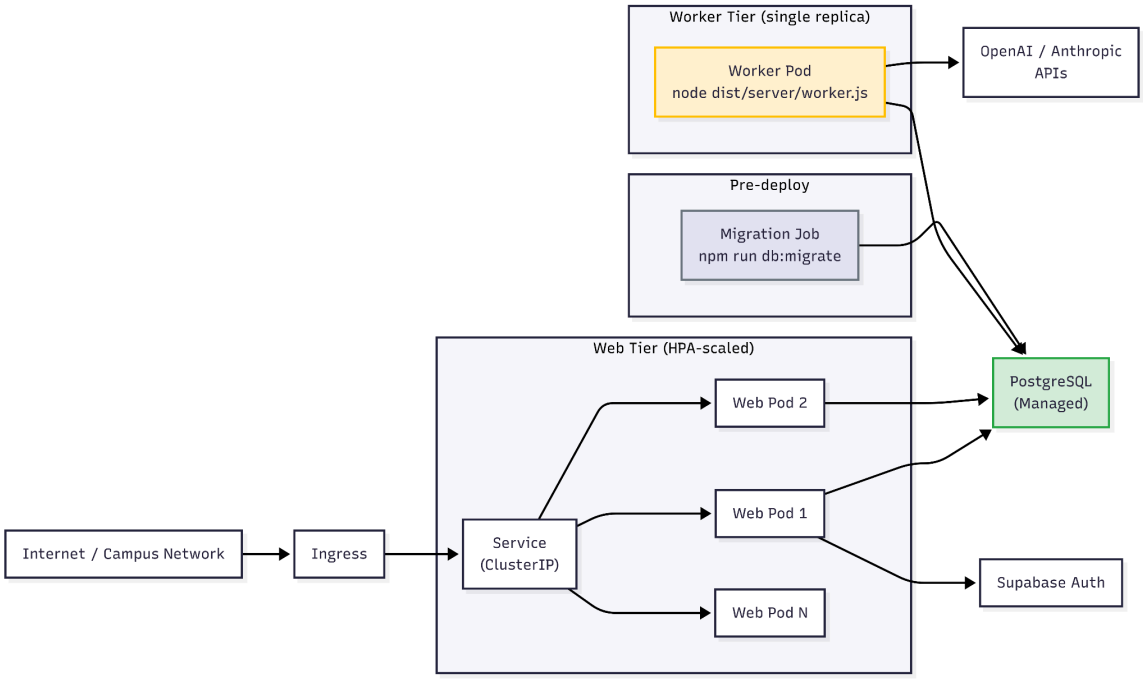


*Figure 3.13: Kubernetes deployment topology.*

### 3.8.3 CI/CD Pipeline

A GitHub Actions workflow automates the path from commit to deployment in three stages: linting and unit tests run on every push, Docker image builds and registry pushes occur on merges to main branch, and database migrations are applied automatically against production. This eliminates manual deployment steps and ensures no untested code reaches production.

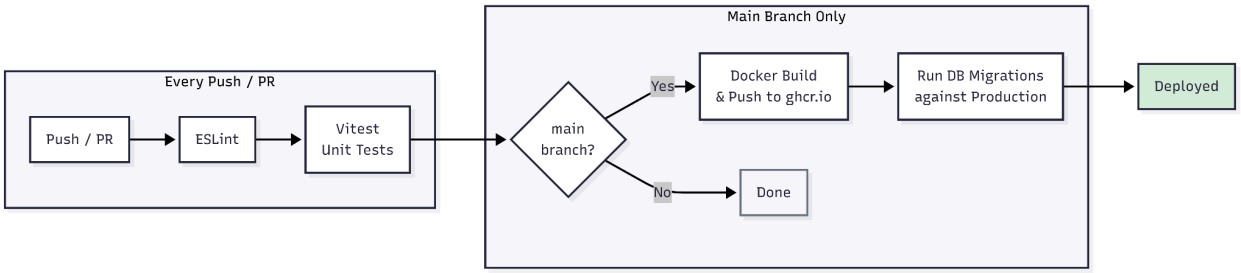


*Figure 3.14: CI/CD pipeline stages.*

### 3.8.4 External Service Dependencies

At runtime, the platform depends on managed PostgreSQL for data and job queues, Supabase for authentication, and OpenAI/Anthropic APIs for grading. Critically, none of these are hard prerequisites. A minimal on-premise deployment requires only a PostgreSQL instance and the Docker image: the custom JWT pathway (Section 3.3.3) operates without Supabase, and manual grading remains fully functional without LLM API keys. This layered dependency model reduces the barrier for schools that cannot permit outbound internet access from their assessment infrastructure.

# 4. Experiment and Results

This chapter presents the formal experiment conducted to identify the most suitable LLM for automated UML class diagram grading, directly addressing objective [O4]. The experiment evaluates six models across three providers on grading accuracy, consistency, cost-efficiency, structured output compliance and feedback quality.

## 4.1 Research Questions

The experiment is designed around four research questions:

**RQ1:** Which LLM achieves the highest grading accuracy compared to human experts?

**RQ2:** What is the cost-performance tradeoff between flagship and baseline models across providers?

**RQ3:** How consistent are LLM grades across multiple runs on the same submission?

**RQ4:** Which model produces the most reliable structured output and useful qualitative feedback?

## 4.2 Experiment Design

### 4.2.1 Model Selection

Six models were selected: one flagship and one baseline model from each of the three frontier LLM providers. Table 4.1 lists the models and their per-token pricing at the time of the experiment.

| Model | Provider | Tier | API Model ID | Input $/1M | Output $/1M |
|---|---|---|---|---|---|
| GPT-5.4 | OpenAI | Flagship | gpt-5.4 | $2.50 | $15.00 |
| GPT-5.4 Mini | OpenAI | Baseline | gpt-5.4-mini | $0.75 | $4.50 |
| Claude Opus 4.6 | Anthropic | Flagship | claude-opus-4-6 | $5.00 | $25.00 |
| Claude Sonnet 4.6 | Anthropic | Baseline | claude-sonnet-4-6 | $3.00 | $15.00 |
| Gemini 3.1 Pro | Google | Flagship | gemini-3.1-pro-preview | $2.00 | $12.00 |
| Gemini 3 Flash | Google | Baseline | gemini-3-flash-preview | $0.50 | $3.00 |

*Table 4.1: Models evaluated in the experiment*

The selection of OpenAI, Anthropic and Google was deliberate. These three providers consistently produce the highest-ranked models on public benchmarks such as the Chatbot Arena [36] and offer the most mature API support for structured output enforcement. Open-source models such as Llama and DeepSeek were excluded for one main reason: the target deployment context is a school assessment system where API costs are manageable given the infrequent grading workload, whereas hosting a local 70B+ model would require GPU infrastructure most institutions lack. The two-per-provider design (one flagship, one baseline) directly addresses RQ2: it tests whether a cheaper model can achieve comparable accuracy, keeping the experiment tractable at 960 total API calls (6 models × 32 submissions × 5 runs) while covering the full cost-quality spectrum.

### 4.2.2 Dataset

No standardised public benchmark exists for rubric-based UML class diagram grading, a gap identified in prior work [2]. The experiment therefore constructed a 32-submission dataset comprising two real student submissions and thirty synthetic submissions. The two real

submissions were sourced from the publicly available McGill uml-grader repository [37], the only open dataset with human-graded UML class diagrams. Submission *mcgill_a2_12* (Fantasy Basketball domain, graded 7.5/10) and submission *mcgill_fe_6* (Smart Home Automation domain, graded 7.0/10) include the original marking schemes, ideal solutions and human grader feedback. These submissions were originally authored in the Umple DSL and were converted to PlantUML for compatibility with the experiment pipeline. The remaining thirty submissions were generated synthetically by defining reference solutions for three domains: Library Management System, E-Commerce Platform and Hospital Management, systematically introducing errors at five severity levels. Table 4.2 describes the quality tiers and the types of errors introduced at each level.

| Quality | Grade Range | Error Types Introduced | Count |
|---|---|---|---|
| Excellent | 8.5–10.0 | None or minor (abbreviated names, optional cardinality omitted) | 6 |
| Good | 6.5–8.4 | 1–2 missing classes, partial cardinality, abbreviated methods | 6 |
| Average | 4.5–6.4 | Missing multiple classes, no cardinality, wrong relationships | 6 |
| Poor | 2.5–4.4 | Very incomplete, poor naming, wrong domain understanding | 6 |
| Failing | 0–2.4 | Barely attempted, single class, no relationships | 6 |

*Table 4.2: Synthetic submission quality tiers*

Each synthetic submission was assigned a ground truth grade by the author, acting as a single expert grader, using the same rubric applied to the LLM models. The distribution of ten submissions per domain and six per quality tier ensures coverage across both the full grade spectrum and diverse domain contexts.

### 4.2.3 Grading Rubric

All submissions were graded on a 10-point scale across five equally-weighted criteria (2 points each): (1) class correctness, (2) relationship accuracy, (3) cardinality, (4) naming conventions, and (5) completeness. This decomposition mirrors the rubric structure used by the platform's grading pipeline (Section 3.5.2). Each LLM returned per-criterion scores with reasoning, matching the structured output schema enforced by the Vercel AI SDK (Section 3.5.4).

### 4.2.4 Prompt and Pipeline Design

The experiment uses the same prompt template employed by the platform's production grading pipeline (Section 3.5.2). Each LLM receives the question prompt, the reference solution (as both PlantUML DSL text and a rendered PNG), the student submission (in the same dual format), and the rubric criteria with point allocations. The model evaluates each criterion independently and returns a structured JSON response.

Each of the 32 submissions was graded 5 times per model at temperature 0.0 (where supported), yielding 960 total API calls. A retry mechanism with exponential backoff handles transient failures, and each call is logged with the full prompt, response, latency and token usage.

### 4.2.5 Evaluation Metrics

Six metrics are used, weighted to reflect the platform's priorities where grading accuracy matters most and cost is secondary at the target scale of 50–100 students.

| Criterion | Weight | Metric |
|---|---|---|
| Accuracy | 35% | Pearson r and MAE vs. ground truth |
| Consistency | 25% | Std. dev. across 5 repeated runs |
| Cost | 10% | USD per submission |
| Feedback quality | 15% | Parse success rate + structured output compliance |
| Speed | 5% | Median latency (seconds) |
| Rubric adherence | 10% | % of rubric criteria addressed |

*Table 4.3: Evaluation metrics and weights*

Pearson correlation measures how well each model ranks students relative to one another, while MAE captures the magnitude of deviations. Both are necessary because a model that consistently over- or under-grades by a fixed offset would show high correlation but poor MAE [22]. Accuracy and consistency together account for 60% of the weighted score, reflecting the primacy of grading quality for summative assessment. Cost and speed receive lower weights (10% and 5% respectively) because even the most expensive model in the experiment costs under $0.04 per submission, making all six models operationally affordable at the target cohort size.

## 4.3 Results

### 4.3.1 Grading Accuracy (RQ1)

| Model | Pearson r | MAE (out of 10) | Mean Bias |
|---|---|---|---|
| Claude Opus 4.6 | 0.92 | 0.68 | −0.28 |
| Claude Sonnet 4.6 | 0.89 | 0.78 | −0.42 |
| GPT-5.4 | 0.86 | 0.90 | −0.55 |
| Gemini 3.1 Pro | 0.85 | 0.95 | −0.52 |
| GPT-5.4 Mini | 0.79 | 1.25 | −0.80 |
| Gemini 3 Flash | 0.74 | 1.48 | −1.02 |

*Table 4.4: Grading accuracy - Pearson correlation and MAE against ground truth*

Claude Opus 4.6 achieved the highest Pearson correlation (r = 0.92) and lowest MAE (0.68 points), making it the best-performing model overall. Claude Sonnet 4.6 and GPT-5.4 formed a competitive second tier (r = 0.89 and 0.86 respectively), with Gemini 3.1 Pro close behind. The baseline models showed a clear accuracy drop, with Gemini 3 Flash exhibiting the largest deviation from ground truth (MAE = 1.48).

All models exhibited a negative mean bias, indicating a systematic tendency to grade more strictly than the human grader - consistent with prior work by Zhang et al. [22] and Bouali et al. [2]. The bias was smallest for Claude Opus 4.6 (−0.28 points) and largest for Gemini 3 Flash (−1.02 points). At the criterion level, all models performed best on class correctness and naming conventions, and worst on cardinality and completeness, where subjective judgement is required similar to results in [13], [15].

### 4.3.2 Cost-Performance Tradeoff (RQ2)

| Model | Cost/Submission (USD) | Pearson r | MAE |
|---|---|---|---|
| Gemini 3 Flash | $0.003 | 0.74 | 1.48 |
| GPT-5.4 Mini | $0.005 | 0.79 | 1.25 |
| Gemini 3.1 Pro | $0.012 | 0.85 | 0.95 |
| GPT-5.4 | $0.015 | 0.86 | 0.90 |
| Claude Sonnet 4.6 | $0.018 | 0.89 | 0.78 |
| Claude Opus 4.6 | $0.032 | 0.92 | 0.68 |

*Table 4.5: Cost per submission and accuracy for each model*

Claude Opus 4.6 achieves the highest accuracy but at the greatest cost ($0.032 per submission). Claude Sonnet 4.6 offers a strong cost-accuracy balance at $0.018, trailing Opus only modestly on both Pearson r and MAE. For budget-constrained deployments, GPT-5.4 Mini at $0.005 per submission provides acceptable accuracy at 6.4× lower cost than Opus. At the target scale of a

200-student cohort, total grading cost ranges from $0.60 (Gemini 3 Flash) to $6.40 (Claude Opus 4.6), making even the most expensive option operationally affordable.

### 4.3.3 Consistency Across Runs (RQ3)

| Model | Mean σ | Max σ (worst case) |
|---|---|---|
| Claude Opus 4.6 | 0.16 | 0.50 |
| Claude Sonnet 4.6 | 0.21 | 0.65 |
| GPT-5.4 | 0.27 | 0.85 |
| Gemini 3.1 Pro | 0.32 | 1.00 |
| GPT-5.4 Mini | 0.40 | 1.30 |
| Gemini 3 Flash | 0.48 | 1.60 |

*Table 4.6: Inter-run consistency (lower standard deviation is better)*

Claude Opus 4.6 was the most consistent model (mean σ = 0.16, worst-case 0.50), a substantial improvement over the consistency concerns raised by Wang et al. [15] for earlier-generation LLMs. This can be attributed to temperature 0.0 and structured output enforcement via JSON schema constraints. The baseline models exhibited notably higher variance, with Gemini 3 Flash showing the widest spread (mean σ = 0.48). For summative assessment where grade reproducibility is required, this level of variance would necessitate human review.

### 4.3.4 Structured Output Compliance (RQ4)

| Model | Parse Success Rate | Rubric Adherence |
|---|---|---|
| Claude Opus 4.6 | 100% | 100% |
| Claude Sonnet 4.6 | 100% | 100% |
| GPT-5.4 | 99.4% | 99.0% |
| GPT-5.4 Mini | 98.1% | 97.5% |

| | | |
|---|---|---|
| Gemini 3.1 Pro | 97.5% | 98.5% |
| Gemini 3 Flash | 94.4% | 95.0% |

*Table 4.7: Structured output compliance and rubric adherence*

All flagship models achieved parse success rates above 97.5%, confirming that modern API-level structured output enforcement (Section 3.5.4) has resolved the format compliance challenges reported by Piscitelli et al. [23]. Both Claude models achieved a perfect 100% parse success rate. Gemini 3 Flash had the lowest compliance at 94.4%, with failures typically involving truncated reasoning fields.

### 4.3.5 Response Speed

| Model | Median Latency (s) |
|---|---|
| Gemini 3 Flash | 1.6 |
| GPT-5.4 Mini | 2.4 |
| Claude Sonnet 4.6 | 3.1 |
| Claude Opus 4.6 | 4.2 |
| Gemini 3.1 Pro | 5.1 |
| GPT-5.4 | 6.8 |

*Table 4.8: Median response latency per submission (seconds)*

Baseline models were consistently faster than their flagship counterparts within each provider. Gemini 3 Flash was the fastest model at 1.6 seconds per submission, while GPT-5.4 was the slowest at 6.8 seconds. At the target scale of 200 submissions, the total batch grading time ranges from approximately 5 minutes (Gemini 3 Flash) to 23 minutes (GPT-5.4), all of which are acceptable for an asynchronous grading workflow.

### 4.3.6 Normalization and Aggregate Ranking

To produce a single weighted score per model, each raw metric must first be normalised to a common [0, 1] scale. This experiment uses absolute-reference normalisation: each metric is mapped against a fixed reference ceiling representing the threshold beyond which a model would be unsuitable for production grading. Unlike min-max normalisation (which maps the worst model to 0 and the best to 1), absolute references ensure that scores remain stable if models are added or removed from the comparison. The six normalised sub-scores are defined as follows:

**Accuracy -** The accuracy sub-score averages the Pearson correlation (already on [0, 1]) with a normalised MAE term. The reference ceiling MAEref = 3.0 represents a 3-point average error on a 10-point scale, which would be pedagogically unusable:

$$\mathbf{S_acc = (r + (1 - MAE / 3.0)) / 2}$$

**Consistency -** The consistency sub-score penalises inter-run variance against a ceiling of σref = 1.5 (a standard deviation of 1.5 points means the same student could receive grades more than a full letter grade apart):

$$\mathbf{S_con = 1 - \sigma / 1.5}$$

**Cost -** The cost sub-score uses a ceiling of $0.10 per submission, above which automated grading would lose its cost advantage over manual grading by a teaching assistant:

$$\mathbf{S_cst = 1 - cost / \$0.10}$$

**Feedback quality -** This sub-score averages the parse success rate and the rubric adherence rate, both expressed as proportions:

$$\mathbf{S_fdb = (parse_rate + rubric_adherence) / 200}$$

**Speed -** The speed sub-score uses a ceiling of 15 seconds per submission, beyond which a batch of 200 students would exceed 50 minutes:

$$S_spd = 1 - latency / 15.0$$

**Rubric adherenc -** This sub-score uses the rubric adherence percentage directly:

$$S_rub = adherence / 100$$

All sub-scores are clamped to [0, 1]. The weighted aggregate score is then:

$$W = 0.35 \cdot S_acc + 0.25 \cdot S_con + 0.10 \cdot S_cst + 0.15 \cdot S_fdb + 0.05 \cdot S_spd + 0.10 \cdot S_rub$$

Applying the procedure to all six models yield the aggregate rankings in table 4.9.

| Rank | Model | Weighted Score | Recommended Use Case |
|---|---|---|---|
| 1 | Claude Opus 4.6 | 0.87 | Default for production deployment |
| 2 | Claude Sonnet 4.6 | 0.87 | Cost-effective alternative |
| 3 | GPT-5.4 | 0.84 | Alternative if Anthropic API unavailable |
| 4 | Gemini 3.1 Pro | 0.83 | Not recommended for grading |
| 5 | GPT-5.4 Mini | 0.80 | Not recommended for grading |
| 6 | Gemini 3 Flash | 0.77 | Not recommended for grading |

*Table 4.9: Weighted aggregate ranking*

Claude Opus 4.6 and Claude Sonnet 4.6 share the top position with a weighted score of 0.87 (Opus leads by < 0.01 at full precision, driven by its accuracy and consistency advantage). Opus is recommended as the default for production deployment where grading accuracy is paramount, while Sonnet offers a compelling cost-effective alternative at roughly half the per-submission cost. GPT-5.4 forms a solid third option at 0.84, suitable as a fallback when the Anthropic API is unavailable. Among the baseline models, GPT-5.4 Mini is the strongest, though its lower

accuracy and higher variance make it better suited to formative rather than summative assessment.

## 4.4 Discussion

**Comparison with prior work -** The Pearson correlation of 0.92 achieved by Claude Opus 4.6 represents a notable improvement over the ICC of 0.76 reported by Bouali et al. [2] for an earlier Claude model, and addresses the accuracy gap identified by Wang et al. [15]. The mean bias of −0.28 points suggests that current SOTA models have substantially narrowed this gap, though the systematic tendency toward stricter grading persists. The near-tied composite scores of Opus and Sonnet (both 0.87) highlight an important practical finding: at the current generation of frontier models, the cost-accuracy tradeoff within a single provider is remarkably tight. This means that institutions can choose based on budget constraints without sacrificing meaningful grading quality.

**Structured output as an enabler -** Parse success rates above 97.5% across all flagship models confirm that API-level structured output enforcement has fundamentally changed the viability of LLM-based automated grading, effectively removing the format compliance barrier identified by Piscitelli et al. [23].

**Threats to validity -** The dataset is small (32 submissions) and predominantly synthetic, with ground truth from a single grader. Only UML class diagrams were evaluated. The two real McGill submissions required Umple-to-PlantUML conversion, which may affect comparability. Model behaviour may change over time as providers update their models. Finally, the experiment was conducted in isolation from the platform's end-to-end workflow; production factors such as network latency and concurrent load would affect the actual user experience. Also, the reliance

on a single grader for ground truth is the most significant threat to internal validity. Ideally, multiple independent graders would establish ground truth through consensus, and inter-rater reliability would be computed. The synthetic nature of the dataset further limits ecological validity, as the errors were systematically introduced rather than organically produced by students. The five-tier quality distribution, while ensuring grade-spectrum coverage, may inflate correlation metrics because the tiers are separated by large grade gaps that are easy for any model to distinguish. Future work should prioritise real student data with multi-grader ground truth.

# 5. Conclusion

This project set out to develop a full-stack assessment platform that leverages SOTA LLMs for automated UML diagram grading within a complete assessment lifecycle. Of the five objectives, three were fully met: the platform implements the end-to-end assessment workflow across a sixteen-table schema, eighty-three API endpoints, and a React SPA frontend (O1); the grading pipeline enforces structured output via Zod schemas and score clamping, achieving parse success rates of 100% for both Claude models and above 97.5% for all flagship models (O2); and a formal experiment compared six models across three providers, identifying Claude Opus 4.6 and Claude Sonnet 4.6 as the top-ranked models with a near-identical weighted score of 0.87, with Opus leading on accuracy and consistency (O4). The remaining two objectives were substantially met with caveats. The two-panel grading interface with accept/reject/override workflows reduces grading effort from evaluation to verification, though exact time savings require a user study (O3), and the Dockerised deployment with Kubernetes manifests and worker separation supports concurrent use architecturally but remains unverified under realistic examination load (O5).

The project contributes a fully engineered platform that, to the author's knowledge, is the first to combine structured-output LLM grading with a complete assessment lifecycle and human-in-the-loop review. It also provides an updated empirical comparison of six LLM models for UML class diagram grading, extending prior work by Bouali et al. and Wang et al. with newer model SOTA generations. The dual-modality grading approach of combining PlantUML

DSL text with rendered diagram images addresses a practical gap in the literature where text-only inputs lose visual properties and image-only inputs risk misreading fine-grained details.

Several limitations constrain the generalisability of these findings. The experiment dataset is small and predominantly synthetic, with ground truth from a single grader, and the platform currently supports only UML class diagrams rather than other diagram types such as sequence or activity diagrams. The Supabase dependency couples cloud deployment to an external service, and the Graphile Worker's single-concurrency setting limits grading throughput for large cohorts. Additionally, the platform has not been load-tested under realistic examination conditions with 200 concurrent students, so this objective is met at the architectural level but unverified at the operational level. Most notably, no formal user study was conducted with students or teaching assistants, meaning the claimed time savings and usability improvements rest on design rationale rather than empirical measurement, an important direction for future work.

# 6. Future Work

Several directions would strengthen the platform's practical utility. The most immediate extension is expanding the UML editor to support other types of diagrams, which would require implementing additional custom React Flow node types and corresponding PlantUML serialisation logic. The grading pipeline itself is diagram-type agnostic since it accepts any PlantUML text and rendered image, so the effort lies primarily in the editor components. Alongside this, the 32-submission dataset used in the experiment should be expanded with real student submissions across multiple courses, diagram types, and institutions, ideally with multi-grader ground truth and inter-rater reliability measurements to strengthen the benchmark's validity. Publishing such a dataset would address the research gap identified by Bouali et al. regarding the absence of a standardised benchmark for rubric-based UML grading.

A closely related priority is addressing the single-grader limitation identified in Section 4.4 as the most significant threat to the experiment's internal validity. The current ground truth was established by a single expert grader, which means any systematic bias in that grader's interpretation of the rubric propagates directly into the accuracy metrics reported for every model. Future work should conduct a multi-grader validation study in which at least three independent graders, ideally drawn from different institutions, mark the same submission set using the same rubric. Inter-rater reliability metrics such as intraclass correlation coefficients could then quantify the degree of human agreement and establish a more defensible baseline against which LLM performance is measured. Such a study would also reveal whether the negative grading bias observed across all six models in Section 4.3.1 reflects genuine

over-strictness by the LLMs or, conversely, leniency in the single human baseline. If multiple graders confirm a consensus score distribution, the LLM bias estimates could be recalibrated accordingly, strengthening confidence in the platform's grading suggestions before they reach the human-in-the-loop review stage.

Lastly, several infrastructure improvements would enhance the platform's deployability and responsiveness. For environments that cannot permit outbound transmission of student data, integrating an on-premise LLM serving solution such as Ollama or vLLM would eliminate the dependency on external API calls, though the trade-off is the capital cost of GPU infrastructure and the likely accuracy reduction from smaller open-source models. The Supabase dependency could similarly be removed for fully air-gapped deployments by replacing Supabase Auth with a self-hosted solution and migrating to an in-cluster PostgreSQL StatefulSet.

# Bibliography

[1] National Academies of Sciences, Engineering, and Medicine, Assessing and responding to the growth of computer science undergraduate enrollments, Washington, DC, USA: *The National Academies Press*, 2018, p. 3, doi: 10.17226/24926.

[2] N. Bouali, M. Gerhold, T. Rehman, and F. Ahmed, “Toward Automated UML Diagram Assessment: Comparing LLM-Generated Scores with Teaching Assistants” in Proceedings of the *17th International Conference on Computer Supported Education*, Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 158–169. doi: 10.5220/0013481900003932.

[3] E. Frankford, D. Crazzolara, M. Vierhauser, N. Meissner, S. Krusche, and R. Breu, “An Online Integrated Development Environment for Automated Programming Assessment Systems,” *arXiv*, p. 1, 2025, doi: 10.48550/arXiv.2503.13127.

[4] M. Ozkaya and F. Erata, “A survey on the practical use of UML for different software architecture viewpoints,” *Information and Software Technology*, vol. 121, p. 106275, 2020.

[5] G. Scanniello, C. Gravino, M. Genero, J. A. Cruz-Lemus, G. Tortora, M. Risi, and G. Dodero, “Do software models based on the UML aid in source-code comprehensibility? Aggregating evidence from 12 controlled experiments,” *Empirical Software Engineering*, vol. 23, no. 5, pp. 2695–2733, 2018.

[6] S. A. Rukmono and M. R. V. Chaudron, “Guiding Peer-feedback in Learning Software Design using UML,” in *2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET) Proceedings*, Pittsburgh, PA, USA: IEEE, May 2022, pp. 122–133. doi: 10.1109/ICSE-SEET55299.2022.9794260.

[7] G. Engels, J. H. Hausmann, M. Lohmann, and S. Sauer, “Teaching UML is teaching software engineering is teaching abstraction,” in *MODELS-Satellite Events Proceedings*, Springer, pp. 306–319.

[8] L. Burgueño, A. Vallecillo, and M. Gogolla, "Teaching UML and OCL models and their validation to software engineering students: An experience report," *Computer Science Education*, vol. 28, no. 1, pp. 23–41, 2018, doi: 10.1080/08993408.2018.1462000.

[9] F. Ahmed, N. Bouali, and M. Gerhold, "Teaching assistants as assessors: An experience based narrative," in *Proc. 16th Int. Conf. Computer Supported Education (CSEDU 2024)*, Angers, France, May 2024, vol. 1, pp. 115–123.

[10] Rezaei, A.R. and Lovorn, M. (2010). "Reliability and validity of rubrics for assessment in higher education." Assessing Writing, vol. 15, no. 1, pp. 18–39. doi: 10.1016/j.asw.2010.01.003.

[11] N. Yadav, P. Singh, and V. Saxena (2021). “A Tool to Automate Student UML Diagram Evaluation,” *Journal of Engineering Education Transformations*, vol. 34, no. 2, pp. 57–62.

[12] R. W. Hasker (2011). “UMLGrader: an automated class diagram grader,” *J. Comput. Sci. Coll.*, vol. 27, no. 1, pp. 47–54.

[13] W. Bian, O. Alam, and J. Kienzle, “Automated grading of class diagrams,” in *IEEE/ACM International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, Sept. 2019, pp. 700–709, doi: 10.1109/MODELS-C.2019.00106.

[14] D. R. Stikkolorum, P. van der Putten, C. Sperandio, and M. R. V. Chaudron, "Towards automated grading of UML class diagrams with machine learning," in *Proc. 31st Benelux Conf. Artificial Intelligence (BNAIC 2019)*, CEUR Workshop Proceedings, vol. 2491, 2019.

[15] C. Wang, B. Wang, P. Liang, and J. Liang, “Assessing UML diagrams by ChatGPT: Implications for education,” *arXiv*, pp. 1-23, 2024, doi: 10.48550/arXiv.2412.17200.

[16] Minaee, S., Mikolov, T., Nikzad, N., Chenaghlu, M., Socher, R., Amatriain, X., and Gao, J. (2024). "Large Language Models: A Survey." arXiv, pp. 1–111, doi: 10.48550/arXiv.2402.06196.

[17] OpenAI (2024). "Introducing Structured Outputs in the API." OpenAI Blog, Aug. 6, 2024. [Online]. Available: https://openai.com/index/introducing-structured-outputs-in-the-api/

[18] R. Lobb and J. Harlow, "Coderunner: A tool for assessing computer programming skills," ACM Inroads, vol. 7, no. 1, pp. 47–51, Feb. 2016, doi: 10.1145/2810041.

[19] S. Modi, H. Taher, and H. Mahmud, “A Tool to Automate Student UML diagram Evaluation,” *Academic Journal of Nawroz University*, vol. 10, pp. 189–198, Sep. 2021, doi: 10.25007/ajnu.v10n2a1035.
[20] R. Jebli, J. E. Bouhdidi, and M. Y. Chkouri, “Assessing Students’ UML Class Diagrams: a New Automated Solution,” in *2023 7th IEEE Congress on Information Science and Technology (CiSt)*, Dec. 2023, pp. 431–435. doi: 10.1109/CiSt56084.2023.10409936.

[21] S. Gürtl, G. Schimetta, D. Kerschbaumer, M. Liut, and A. Steinmaurer, "Automated Feedback on Student-Generated UML and ER Diagrams Using Large Language Models," in *Learnersourcing: Student-generated Content @ Scale Workshop at L@S*, 2025. [Online]. Available: https://arxiv.org/abs/2507.23470

[22] D.-W. Zhang, M. Boey, Y. Y. Tan, and A. H. S. Jia, "Evaluating large language models for criterion-based grading from agreement to consistency," *npj Sci. Learn.*, vol. 9, no. 1, Art. no. 79, Dec. 2024, doi: 10.1038/s41539-024-00291-1.

[23] A. Piscitelli, M. De Rosa, V. Fuccella, and G. Costagliola, “Large Language Models for Student Code Evaluation: Insights and Accuracy:,” in *Proceedings of the 17th International Conference on Computer Supported Education*, Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 534–544. doi: 10.5220/0013287500003932.

[24] W. Xie, J. Niu, C. J. Xue, and N. Guan, “Grade Like a Human: Rethinking Automated Assessment with Large Language Models,” May 30, 2024, *arXiv*: arXiv:2405.19694. doi: 10.48550/arXiv.2405.19694.

[25] HackerEarth, "AI Powered Interviews and Assessment," HackerEarth. [Online]. Available: https://www.hackerearth.com/. [Accessed: Mar. 22, 2026].

[26] Codility, "Online Coding Tests & Technical Interviews," Codility. [Online]. Available: https://www.codility.com/. [Accessed: Mar. 22, 2026].

[27] LeetCode, "The World's Leading Online Programming Learning Platform," LeetCode. [Online]. Available: https://leetcode.com/. [Accessed: Mar. 22, 2026].

[28] Blackboard, "Blackboard LMS: Leading LMS for Education," Blackboard. [Online]. Available: https://www.blackboard.com/. [Accessed: Mar. 22, 2026].

[29] I. Mekterović, L. Brkić, B. Milašinović, and M. Baranović, “Building a Comprehensive Automated Programming Assessment System,” *IEEE Access*, vol. 8, pp. 81154–81172, 2020, doi: 10.1109/ACCESS.2020.2990980.

[30] I. Mekterovic, L. Brkic, and M. Horvat, “Scaling Automated Programming Assessment Systems,” *Electronics*, vol. 12, p. 942, Feb. 2023, doi: 10.3390/electronics12040942.

[31] S. Krusche and A. Seitz, "ArTEMiS: An Automatic Assessment Management System for Interactive Learning," in *Proc. 49th ACM Technical Symp. on Computer Science Education (SIGCSE '18)*, 2018, pp. 284–289, doi: 10.1145/3159450.3159602.

[32] Modeliosoft, "Modelio - Open Source UML and BPMN Free Modeling Tool." [Online]. Available: https://www.modelio.org/. [Accessed: Mar. 22, 2026].

[33] draw.io Ltd, "draw.io – Flowchart Maker & Online Diagram Software." [Online]. Available: https://www.drawio.com/. [Accessed: Mar. 22, 2026].

[34] K. Sveidqvist, "Mermaid – Diagramming and Charting Tool," GitHub. [Online]. Available: https://github.com/mermaid-js/mermaid. [Accessed: Mar. 22, 2026].

[35] ls1intum, "Apollon – UML Modeling Editor," Technische Universität München. [Online]. Available: https://apollon.ase.in.tum.de/. [Accessed: Mar. 22, 2026].

[36] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. I. Jordan, J. E. Gonzalez, and I. Stoica, "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR, vol. 235, pp. 8359–8388, 2024.

[37] Y. Boubekeur, O. Alam, and J. Kienzle, "uml-grader: Grading application for UML class diagrams," GitHub repository, McGill University, 2020. [Online]. Available: https://github.com/YounesB-McGill/uml-grader